



Embedded Software for the Internet Of Things

Analog to Digital Converter (ADC)

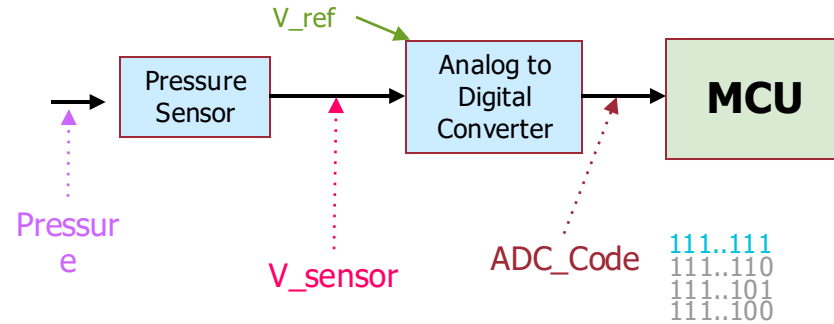
Assoc. Prof. Kasim Sinan Yildirim
University of Trento, Italy

Analog to Digital Conversion

- Embedded systems often need to **measure** values of physical parameters
- These parameters are usually **continuous (analog)** and not in a digital form which **computers** (which operate on **discrete data** values) can process
- Sensor examples:
 - Temperature, Light (or infrared or ultraviolet) intensity, Pressure, Blood pressure, Acceleration

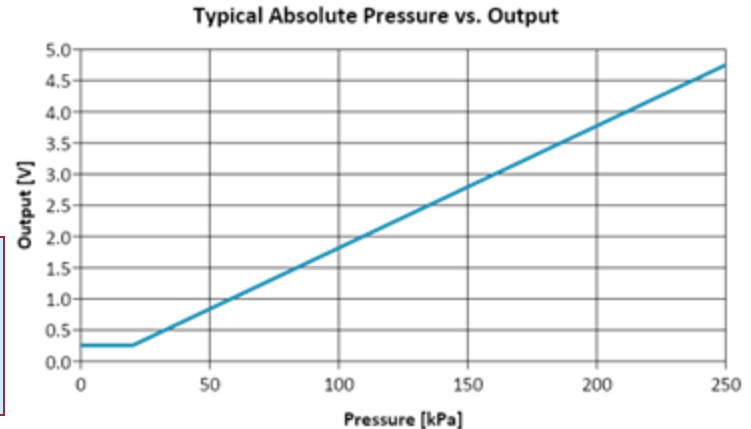
Example Analog Sensor - Depth Gauge

- We calculate how deep underwater an object is:
 - Sensor detects **pressure** and generates a **proportional output voltage V_{sensor}**
 - Analog to Digital Converter generates a proportional digital integer (code) based on **V_{sensor}** and **V_{ref}**



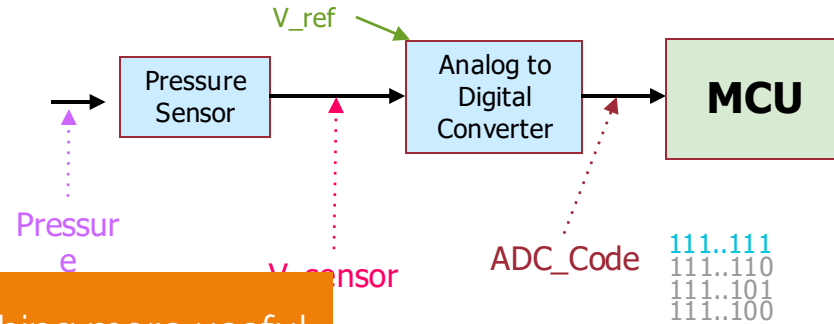
Software on MCU

```
// Your software
ADC_Code = adc_read();
V_sensor = ADC_Code * V_ref / ADC_MASK;
Pressure_kPa = 250 * (V_sensor / V_supply + 0.04);
Depth_ft = 33 * (Pressure_kPa - Atmos_Press_kPa) / 101.3;
```



Example Analog Sensor - Depth Gauge

- We calculate how deep underwater an object is:
 - Sensor detects **pressure** and generates a **proportional output voltage V_{sensor}**
 - Analog to Digital Converter generates a proportional digital integer (code) based

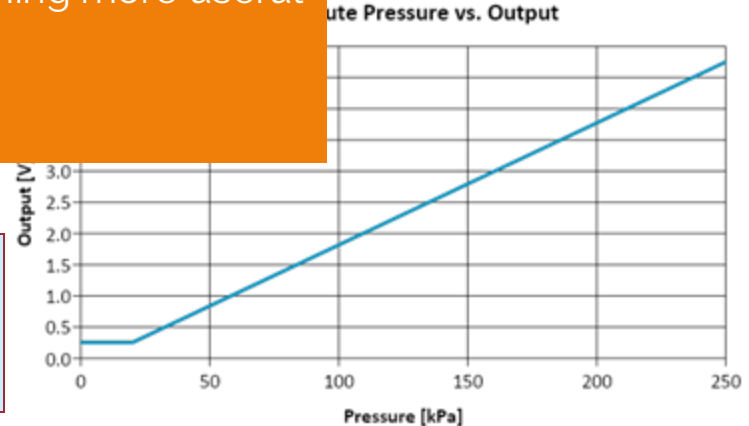


Code can convert that **ADC_Code (integer)** to a something more useful

- **V_{sensor}** : representing the voltage,
- **Pressure_kPa**: representing the pressure
- **Depth_ft**: representing the depth

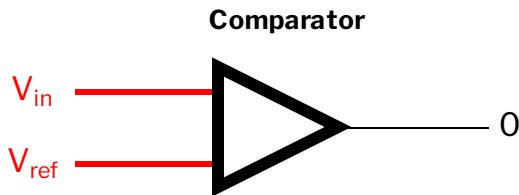
Software on MCU

```
// Your software
ADC_Code = adc_read();
V_sensor = ADC_code * V_ref / ADC_MASK;
Pressure_kPa = 250 * (V_sensor / V_supply + 0.04);
Depth_ft = 33 * (Pressure_kPa - Atmos_Press_kPa) / 101.3;
```



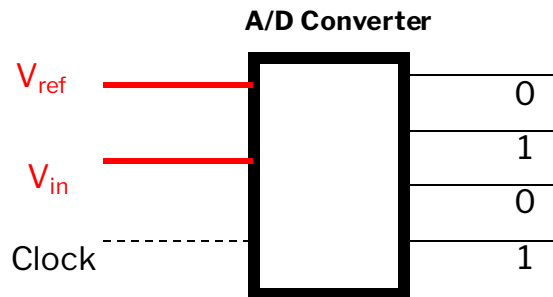
Getting From Analog to Digital

- A **Comparator** tells us “Is $V_{in} > V_{ref}$?”
 - Compares an analog input voltage with an analog reference voltage and determines which is larger, returning a **1-bit number**
 - *E.g. Indicate if depth > 100ft*
 - Set **V_{ref}** to voltage pressure sensor returns with 100 ft depth.

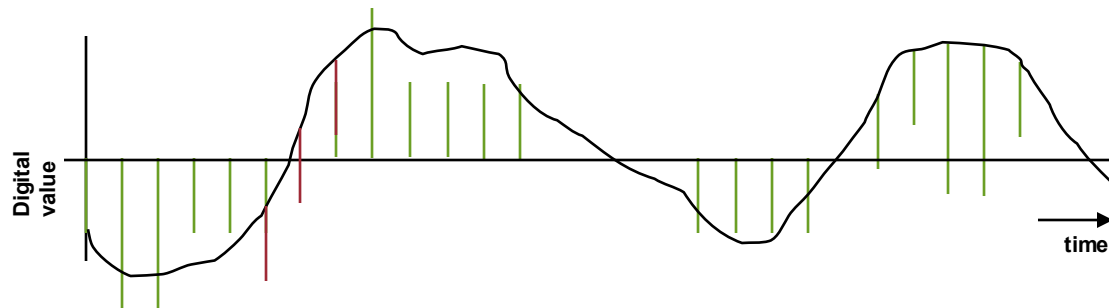


Getting From Analog to Digital

- An **Analog to Digital converter [AD or ADC]** tells us **how large** V_{in} is **as a fraction** of V_{ref} .
 - Reads an analog input signal (usually a voltage) and produces a corresponding **multi-bit number** at the output.
 - *E.g. calculate the depth*

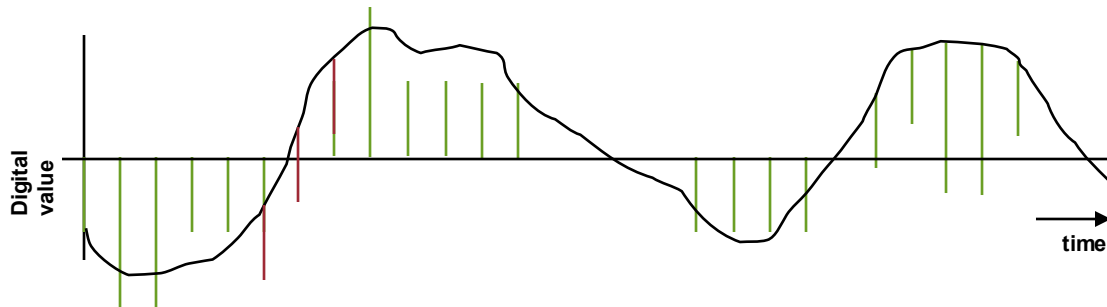


Waveform Sampling and Quantization



- A waveform is **sampled** at a **constant rate** – every Δt
 - Each such sample represents the instantaneous **amplitude** at the instant of sampling
 - “At 37 ms, the input is 1.91341914513451451234311... V”
 - Sampling converts a **continuous time signal** to a **discrete time signal**

Waveform Sampling and Quantization



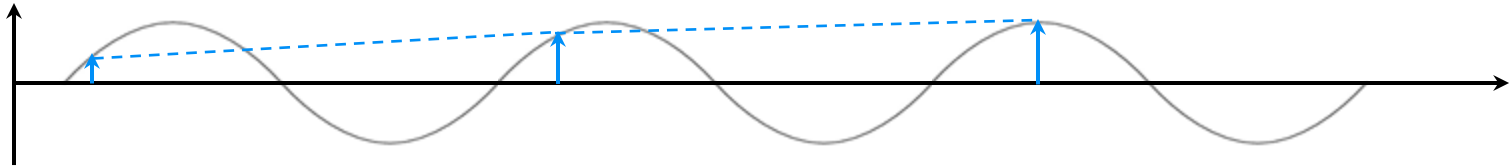
- The sample can now be **quantized** (converted) into a **digital value**
 - Quantization represents a continuous (analog) value with the closest discrete (digital) value
 - "The sampled input voltage of 1.91341914513451451234311... V is best represented by the **code** 0x018, since it is in the range of 1.901 V to 1.9980 V which corresponds to code 0x018."

Sampling Frequency

- When sampling varying signals there is an upper limit on the bandwidth of the converter which is set by the '**Nyquist criterion**'
 - Each cycle of the waveform must be sampled a minimum of twice,

$$F_{\text{sample}} \geq 2 * F_{\text{max frequency component}}$$

- Frequency components above $\frac{1}{2} F_{\text{sample}}$ are aliased, distorting the measured signal



Output Equations

- What code **n** will the ADC use to represent voltage V_{in} ?

$$n = \left\lfloor \frac{V_{in} - V_{-ref}}{V_{+ref} - V_{-ref}} 2^N + 1/2 \right\rfloor$$

$\lfloor X \rfloor = \text{floor}(X)$ *floor(X) **nearest integer** I such that $I \leq X$*
*floor(x+0.5) rounds x to the **nearest integer***

Upper and **lower** reference voltage, and the measured voltage must lie between the two

V_{in} = sampled input voltage

V_{+ref} = upper voltage reference

V_{-ref} = lower voltage reference

N = number of bits of resolution in ADC

Output Equations

- What code **n** will the ADC use to represent voltage V_{in} ?

$$n = \left\lfloor \frac{V_{in}}{V_{+ref}} 2^N + 1/2 \right\rfloor \quad \text{Simplification with } V_{-ref} = 0 \text{ V}$$

$\lfloor X \rfloor = \text{floor}(X)$ *floor(X) nearest integer I such that $I \leq X$*
floor(x+0.5) rounds x to the nearest integer

Upper reference voltage, and the measured voltage must lie between 0 and upper reference

V_{in} = sampled input voltage

V_{+ref} = upper voltage reference

N = number of bits of resolution in ADC

Output Equations

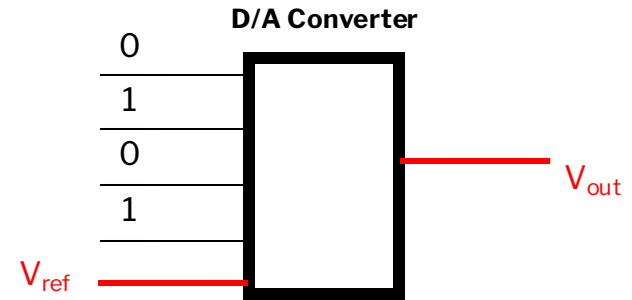
- What code **n** will the ADC use to represent voltage V_{in} ?

$$n = \left\lfloor \frac{V_{in}}{V_{+ref}} 2^N + 1/2 \right\rfloor \quad \text{Simplification with } V_{-ref} = 0 \text{ V}$$

$$n = \left\lfloor \frac{3.3\text{V}}{5\text{V}} 2^{10} + 1/2 \right\rfloor = 388$$

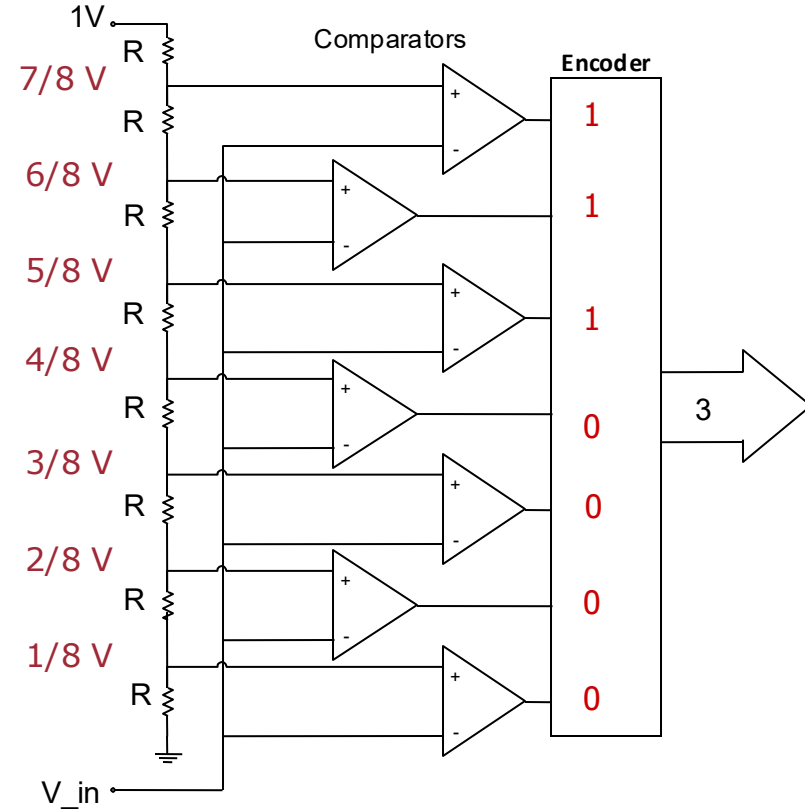
Digital to Analog Conversion

- DAC: “Generate the analog voltage which is this **fraction** of Vref”
 - Digital to Analog Converter equation
 - $V_{out} = V_{ref} * n/(2N)$ or $V_{out} = V_{ref} * (n+1)/(2N)$
 - n = input code
 - N = number of bits of resolution of converter
 - V_{ref} = reference voltage
 - V_{out} = output voltage. Either



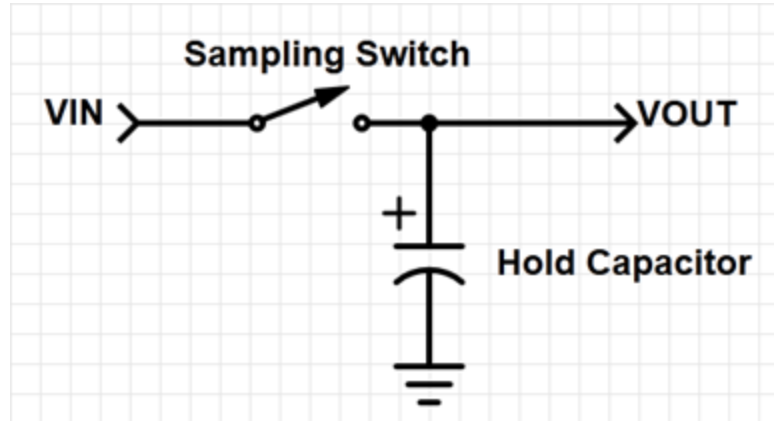
A/D Circuit – Flash Converter

- A multi-level **voltage divider** is used to set voltage levels over the complete range of conversion.
 - A comparator is used at each level to determine whether the voltage is lower or higher than the level.
- The series of comparator outputs are **encoded** to a binary number in digital logic (a priority encoder)



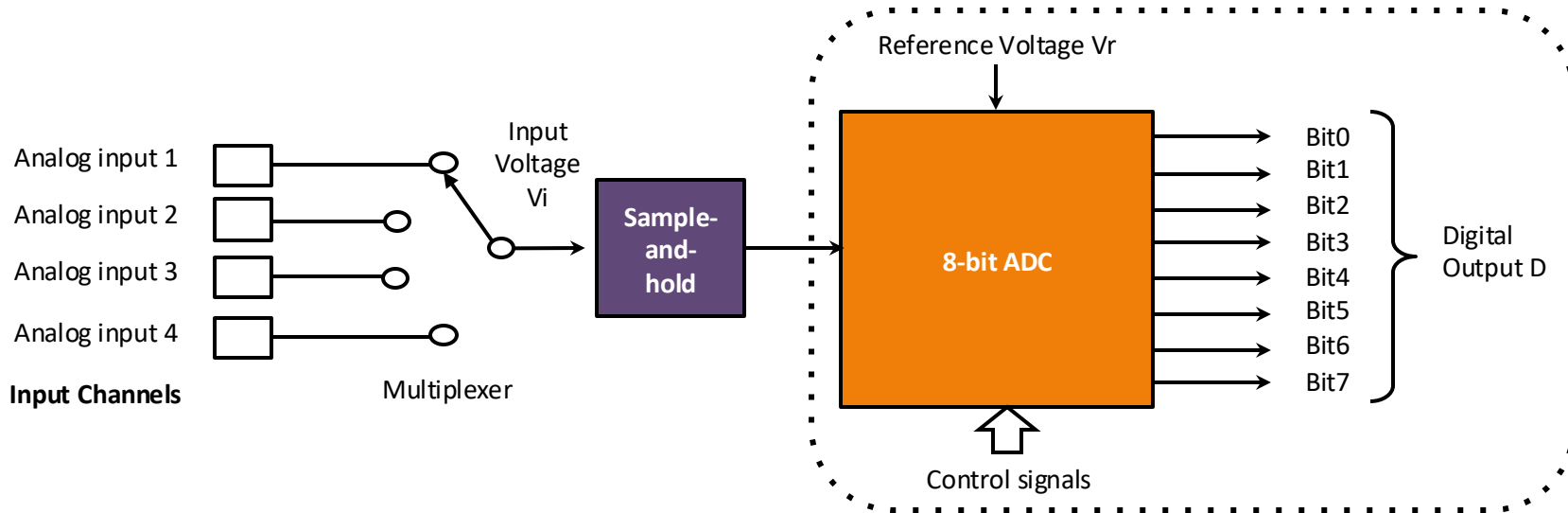
Sample and Hold Devices

- Some A/D converters require the input analog signal to be held **constant** during conversion
 - This function is accomplished by a sample and hold device
 - The capacitor '**holds**' the input voltage until a conversion has been completed.



Analog-Digital-Conversion-System

- An ADC-System usually consists of three stages:
 - **Multiplexer:** reduces the number of required ADC components
 - **Sample-and-hold:** keeps the analog signal during the conversion constant.
 - *Time discretization!*
 - **ADC:** converts the analog into digital signal

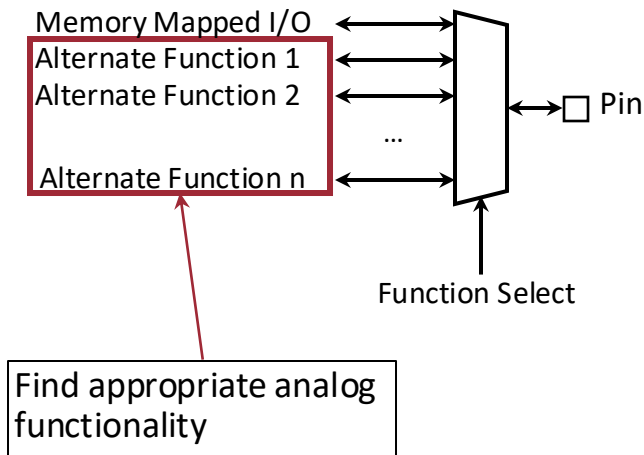


Some Performance Metrics

- **Number of bits** determines overall accuracy.
 - There are always limitations and inaccuracies present in the conversion from Analog to Digital.
- **Conversion time:** between start of conversion and generation of result.
 - Conversion rate = inverse of conversion time.

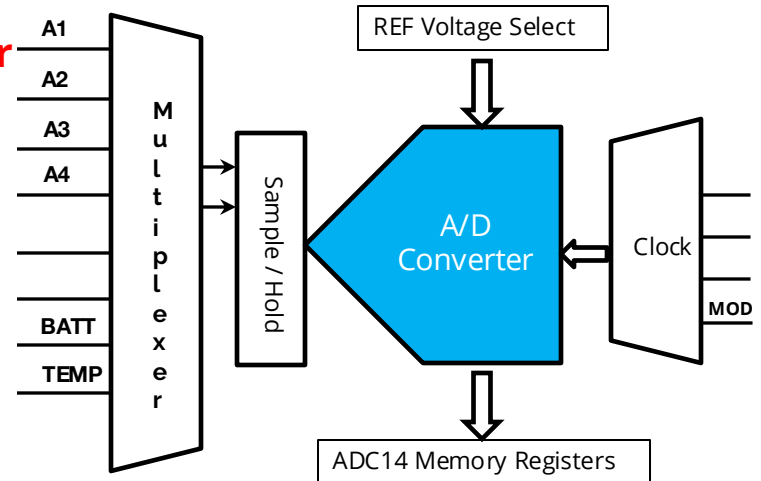
GPIO Alternative Functions

- Pins may have different features
 - may also have analog paths for ADC / DAC etc.
 - to enable an alternative function, set up the appropriate register

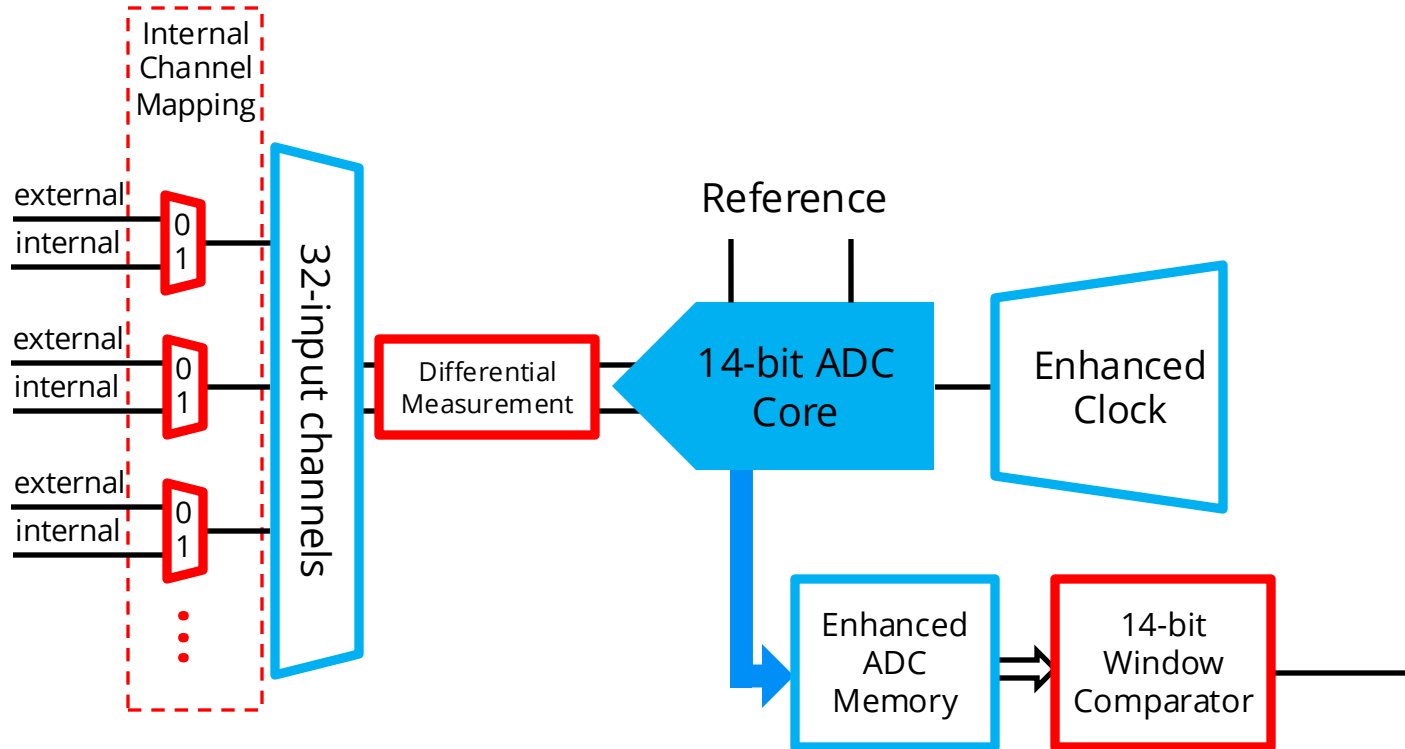


MSP432 ADC Overview

- Analog to Digital Converter
 - **14-bit Accuracy**
 - 32-input channels
 - Separate internal channel for **TempSensor**

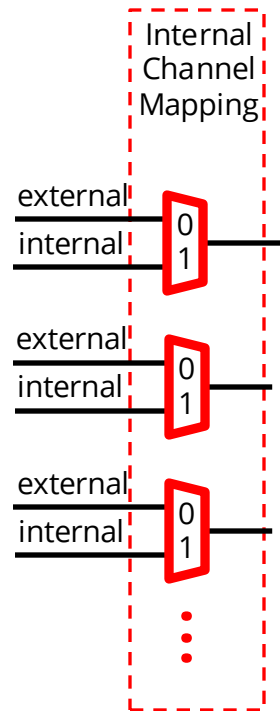


MSP432 ADC Overview



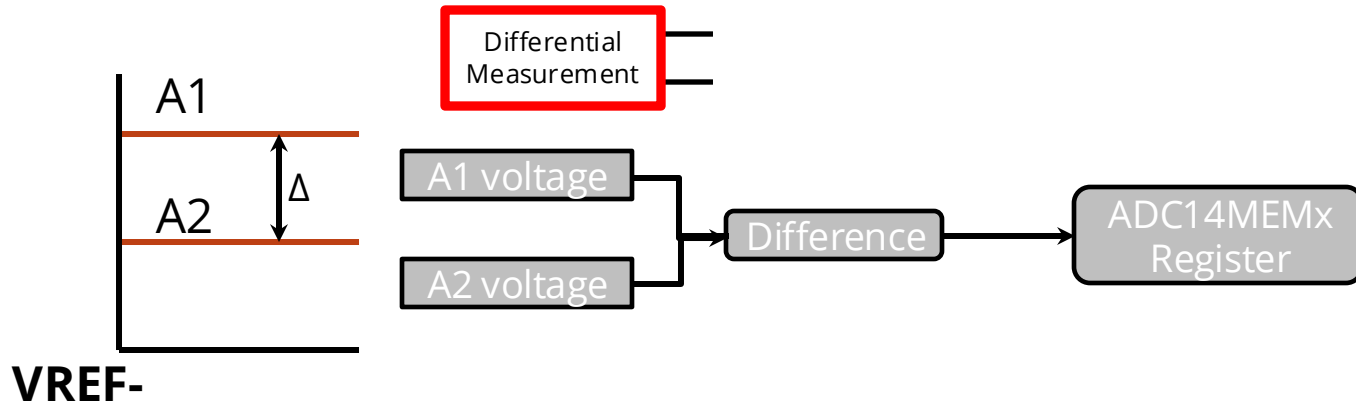
Channels

- A26-31 can map to either an External Input or Internal ADC input
 - Internal inputs include the **temperature sensor** and **battery monitor**.



Differential Measurement

- Combine input channels to create differential input channels
 - ADC will measure the **difference** between two channels and store this value in the ADC14MEMx register

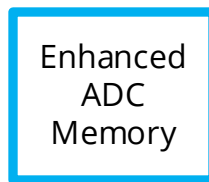


ADC Clock and Memory

- Clock sources, including:
 - MODOSC = 25MHz
 - SYSOSC = 5MHz
- 32 ADC14MEMx registers



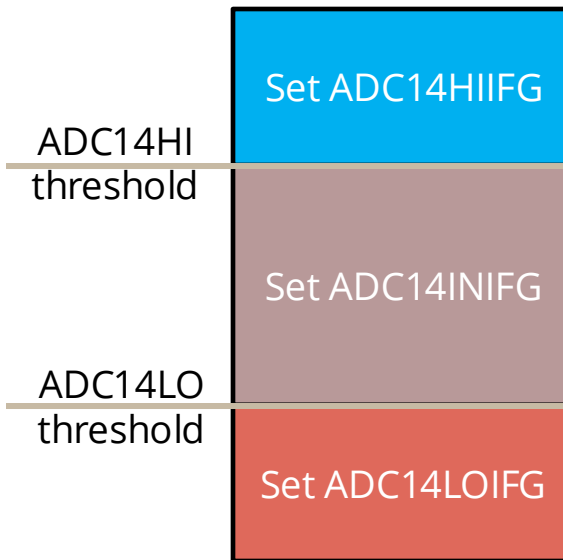
Enhanced
Clock



Enhanced
ADC
Memory

MSP432 ADC Overview

- Configurable input threshold levels
- Conversion results automatically compared against the thresholds
- Hi, Lo, and In interrupts indicate which range the result falls in
 - Same thresholds shared among all channels
 - Useful for low power because device can stay in sleep mode until result falls in window



MSP432 ADC Overview



MSP432 ADC Overview

ADC14HI threshold —————

ADC14LO threshold —————

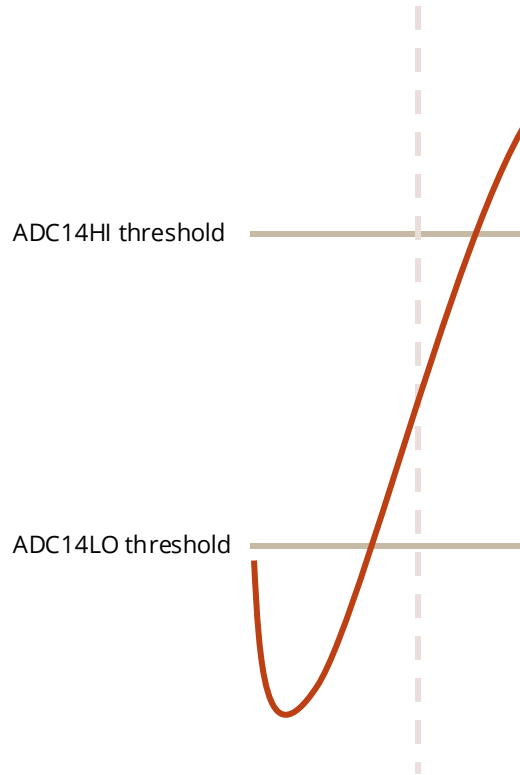


Set ADC14HIFG

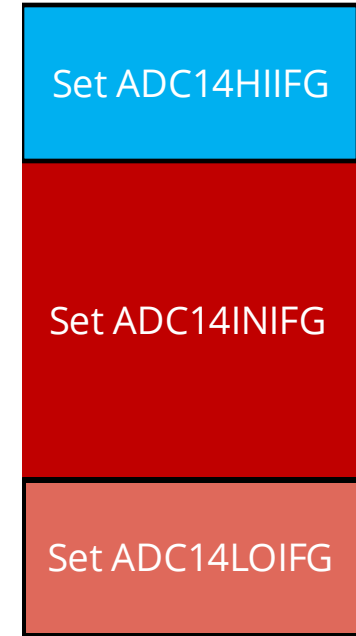
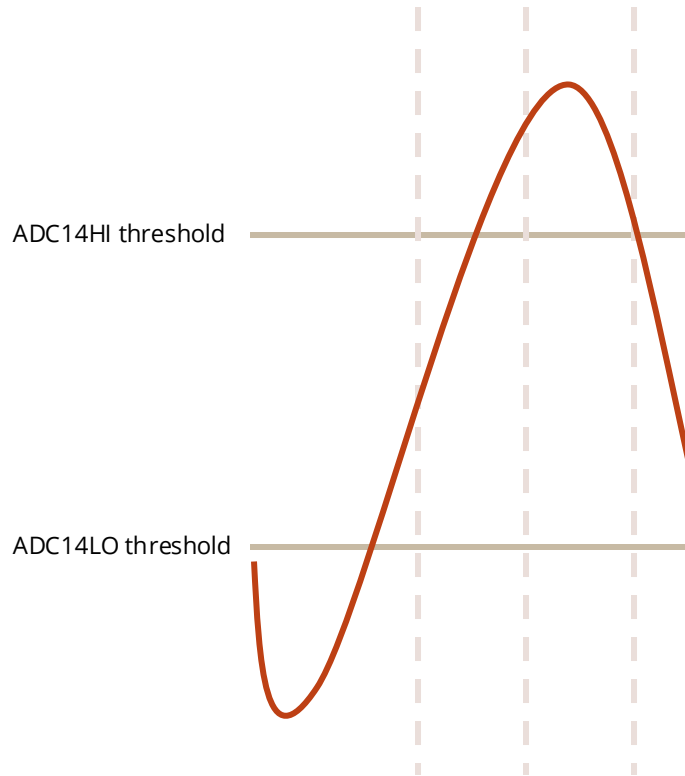
Set ADC14INIFG

Set ADC14LOIFG

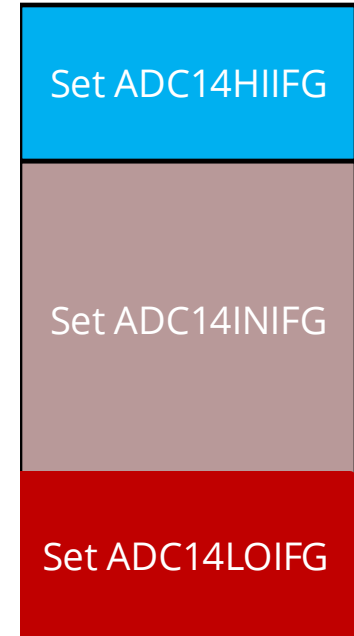
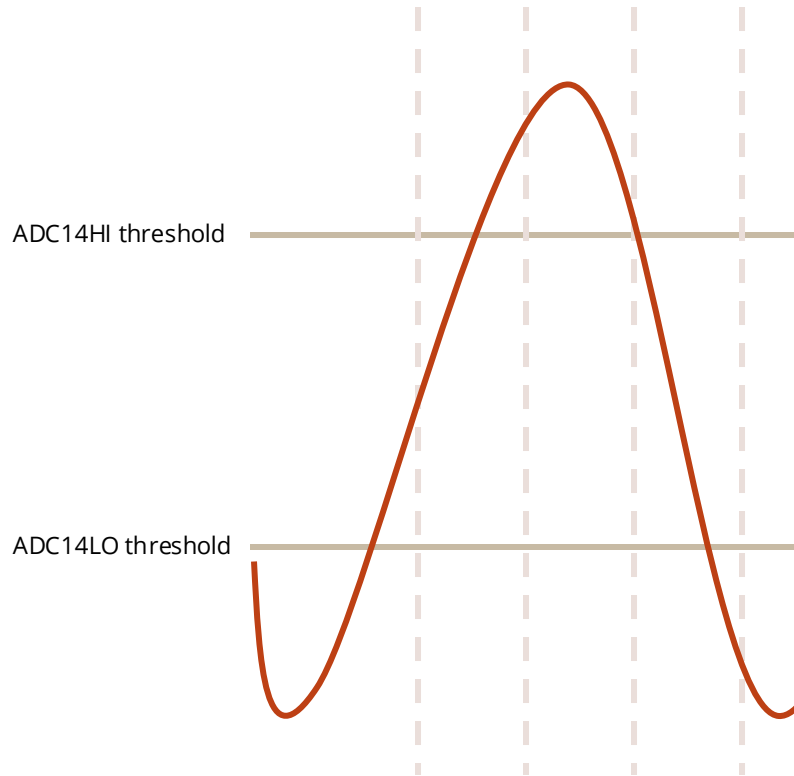
MSP432 ADC Overview



MSP432 ADC Overview



MSP432 ADC Overview

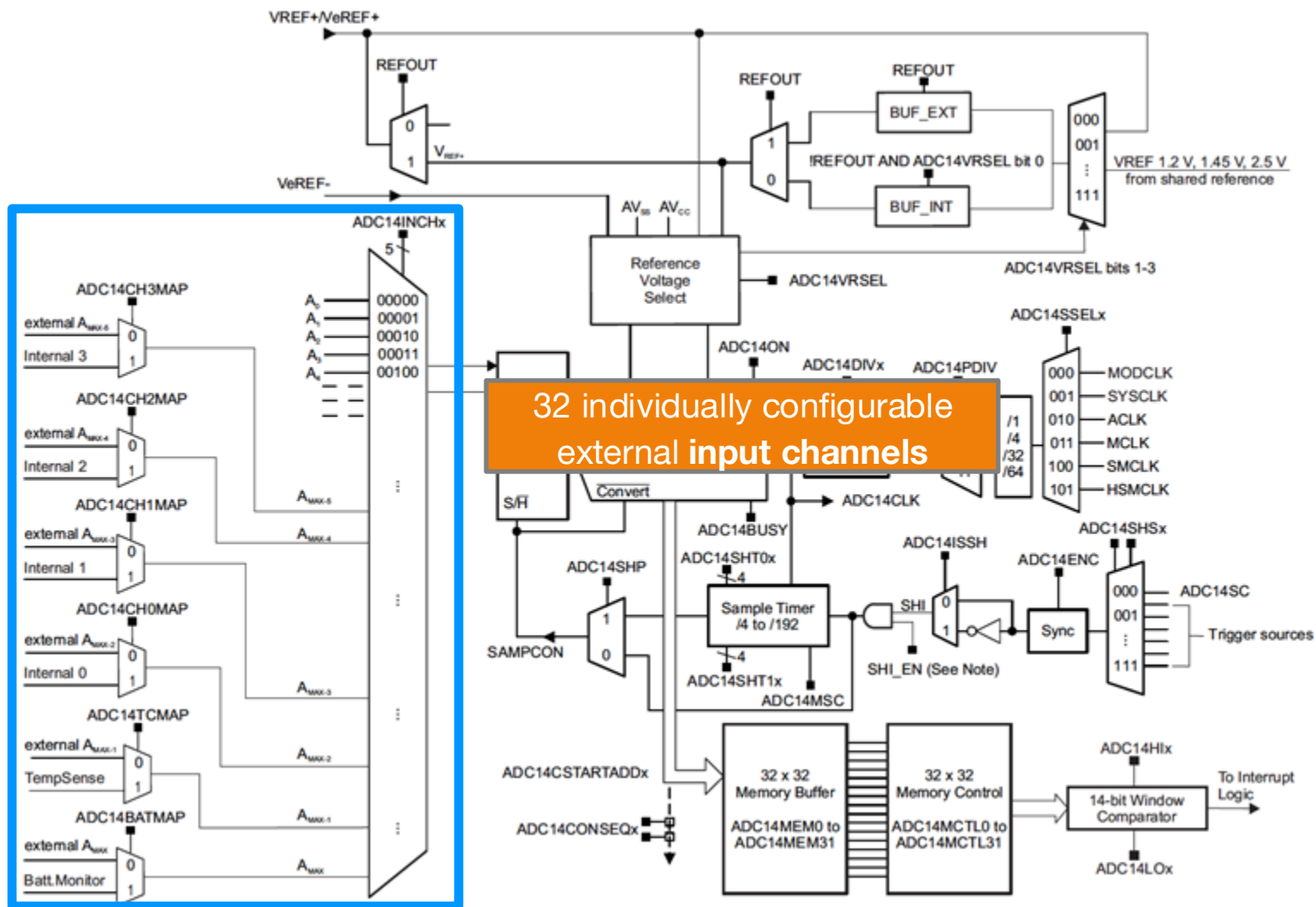


The diagram illustrates the internal architecture of the ADC14 module. Key components and their interconnections include:

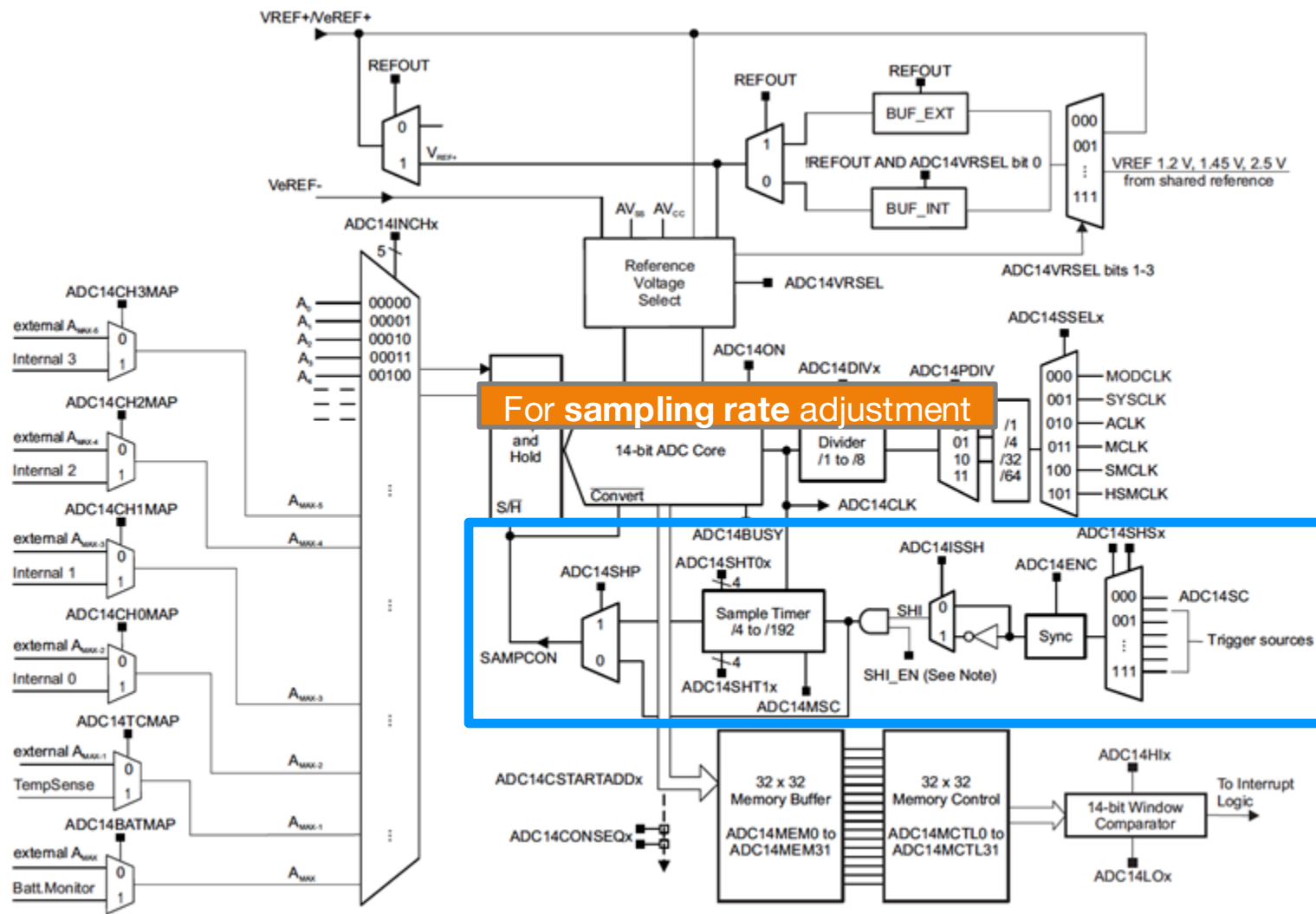
- Reference Voltage Select:** Receives V_{REF+} and V_{REF-} inputs. It selects the reference voltage for the 14-bit ADC Core via AV_{DD} and AV_{SS} . It also provides the $ADC14VRSEL$ signal to the $ADC14SSELx$ multiplexer.
- 14-bit ADC Core:** The central conversion unit. It receives the selected reference voltage and the $ADC14INCHx$ input. It outputs the digital result to the $ADC14BUSY$ signal and the $ADC14SHT0x$ to $ADC14SHT1x$ range.
- Sample and Hold:** Receives the $ADC14INCHx$ input and provides the $SAMPCON$ signal to the $ADC14SHP$ multiplexer.
- Sample Timer:** Receives the $ADC14SHT0x$ to $ADC14SHT1x$ range and provides the $ADC14MSC$ signal to the $ADC14CONSEQx$ multiplexer.
- Memory Buffer and Control:** The $ADC14CONSEQx$ multiplexer selects the sequence of conversions. The $ADC14STARTADDx$ multiplexer selects the starting address in the 32×32 Memory Buffer (ADC14MEM0 to ADC14MEM31). The 32×32 Memory Control (ADC14MCTL0 to ADC14MCTL31) manages the data flow.
- Output and Comparison:** The $ADC14Hix$ multiplexer selects the output data. The 14 -bit Window Comparator (ADC14LOx) compares the output against a threshold and sends a signal to the interrupt logic.
- Control and Status:** The $ADC14SHSx$ multiplexer selects the source for the $ADC14SHS$ signal. The $ADC14ENC$ multiplexer selects the enable signal for the $ADC14SHS$ signal. The $ADC14SHS$ signal is used to enable the $ADC14CONSEQx$ multiplexer.
- Dividers and Multiplexers:** The $ADC14DIVx$ and $ADC14PDIV$ multiplexers select the division factors for the $ADC14CLK$ and $ADC14SHS$ signals, respectively. The $ADC14SSELx$ multiplexer selects the source for the $ADC14SSEL$ signal.

Software-selectable on-chip
reference voltage generation
(1.2 V, 1.45 V, or 2.5 V)

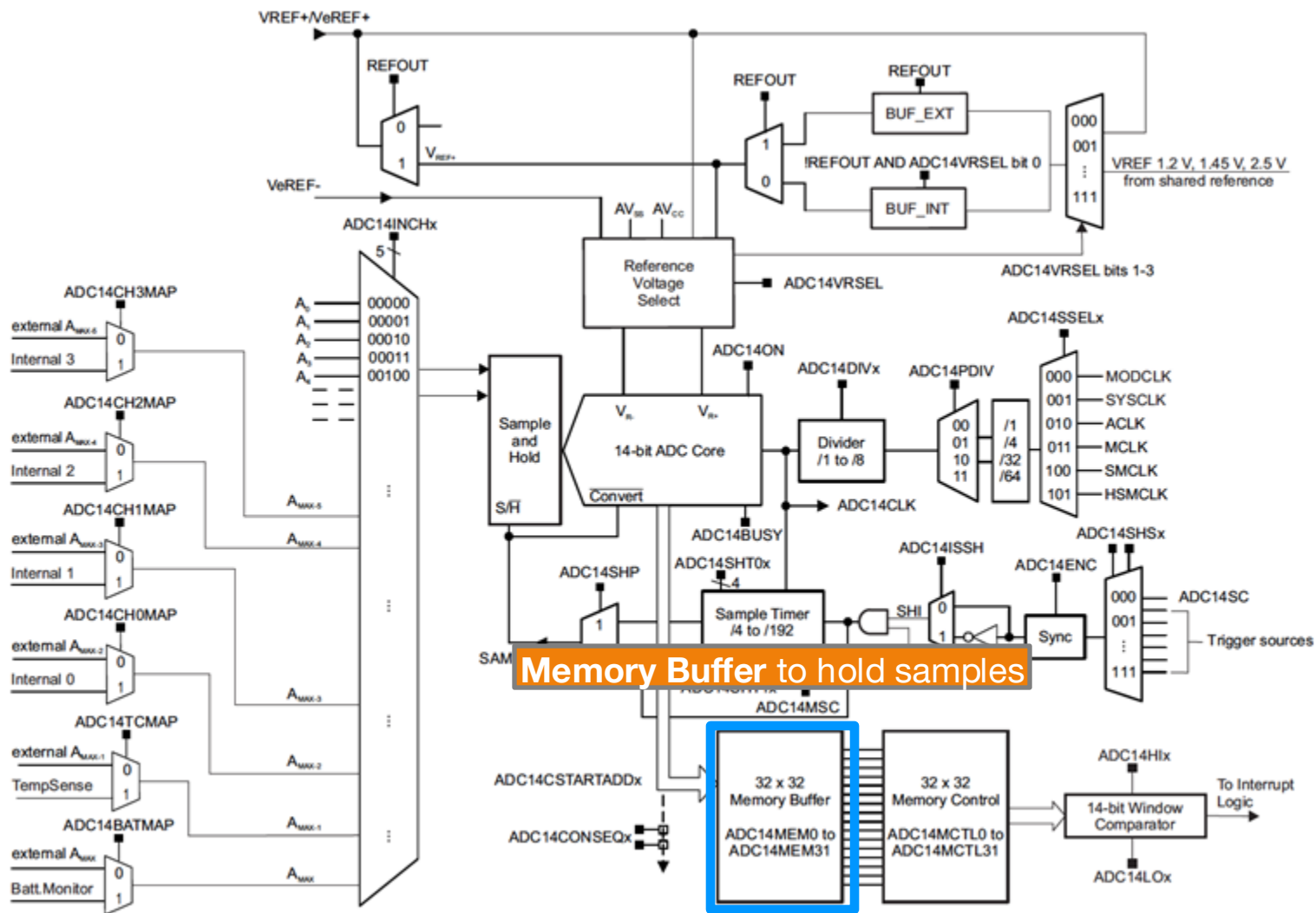
MSP432 ADC



MSP432 ADC



MSP432 ADC



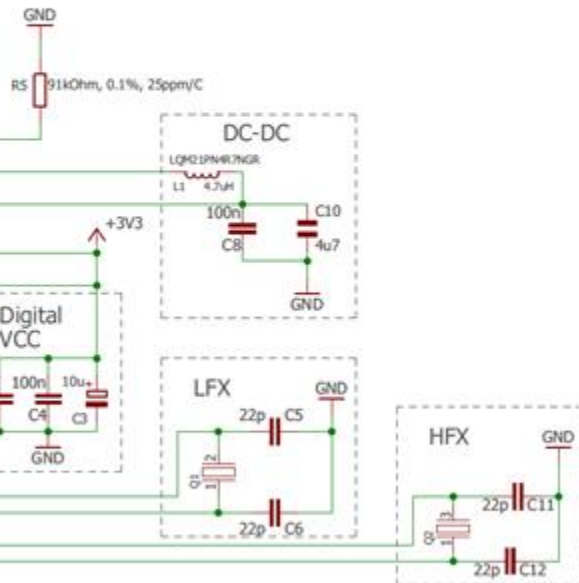
Pins

P1.0_LED1	4	P1.0/UCAGSTE	4
P1.1_BUTTON1	5	P1.1/UCACLK	5
P1.2_BCLUART_RXD	6	P1.2/UCAR0XD/UCAS0SMI	6
P1.3_BCLUART_TXD	7	P1.3/UCAT0XD/UCAS0SMI	7
P1.4_BUTTON2	8	P1.4/UCB0STE	8
P1.5_SPICLK_J1.7	9	P1.5/UCB0CLK	9
P1.6_SPIMOSI_J2.15	10	P1.6/UCB0SIMQ/UCB0SDA	10
P1.7_SPIMISO_J2.14	11	P1.7/UCB0SOMI/UCB0SCL	11
P2.0_RGBLED_RED	16	P2.0/PM_UCA1STE	16
P2.1_RGBLED_GREEN	17	P2.1/PM_UCA1CLK	17
P2.2_RGBLED_BLUE	18	P2.2/PM_UCA1RXD/PM_UCA1SOMI	18
P2.3_IO_J4.34	19	P2.3/PM_UCA1TXD/PM_UCA1SIMO	19
P2.4_PWM_J4.38	20	P2.4/PM_TA0.1	20
P2.5_PWM_J2.19	21	P2.5/PM_TA0.2	21
P2.6_PWM_J4.39	22	P2.6/PM_TA0.3	22
P2.7_PWM_J4.40	23	P2.7/PM_TA0.4	23
P3.0_IO_J2.18	32	P3.0/PM_UCA2STE	32
P3.1	33	P3.1/PM_UCA2CLK	33
P3.2_URXD_J1.3	34	P3.2/PM_UCA2RXD/PM_UCA2SOMI	34
P3.3_UTXD_J1.4	35	P3.3/PM_UCA2TXD/PM_UCA2SIMO	35
P3.4	36	P3.4/PM_UCB2STE	36
P3.5_IO_J4.32	37	P3.5/PM_UCB2CLK	37
P3.6_IO_J2.11	38	P3.6/PM_UCB2SIMQ/PM_UCB2SDA	38
P3.7_IO_J4.31	39	P3.7/PM_UCB2SOMI/PM_UCB2SCL	39
P4.0_A13_J3.24	56	P4.0/A13	56
P4.1_IO_J1.5	57	P4.1/A12	57
P4.2_A11_J3.25	58	P4.2/AGLK/TA2CLK/A11	58
P4.3_A10_J1.6	59	P4.3/MCLK/RTCCLK/A10	59
P4.4_A9_J3.26	60	P4.4/HSMCLK/SVMHCLK/A9	60
P4.5_A8_J3.27	61	P4.5/A8	61
P4.6_IO_J1.8	62	P4.6/A7	62
P4.7_A6_J3.28	63	P4.7/A6	63
P5.0_IO_J2.13	64	P5.0/A5	64
P5.1_IO_J4.33	65	P5.1/A4	65
P5.2_IO_J2.12	66	P5.2/A3	66
P5.3	67	P5.3/A2	67
P5.4_IO_J3.29	68	P5.4/A1	68
P5.5_IO_J3.30	69	P5.5/A0	69
P5.6_BUTTON_J4.35	70	P5.6/UCB1STE/TA2CLK/CL.7	70
P5.7_IO_J2.17	71	P5.7/TA2.2/BEEF-NBEEF/CL.6	71
P6.0_A15_J1.2	54	P6.0/A15	54
P6.1_A14_J3.23	55	P6.1/A14	55
P6.2	56	P6.2	56
P6.3	77	P6.3/UCB1CLK/CL.4	77
P6.4_I2CSDA_J1.10	78	P6.4/UCB1SIMQ/UCB1SDA/CL.3	78
P6.5_I2CSCL_J1.9	79	P6.5/UCB1SOMI/UCB1SCL/CL.2	79
P6.6_CAPTURE_J4.36	80	P6.6/TA2.3/UCB3SIMQ/UCB3SDA/CL.1	80
P6.7_CAPTURE_J4.35	81	P6.7/TA2.4/UCB3SOMI/UCB3SCL/CL.0	81
P7.0	88	P7.0/PM_SMCLK/PM_DMAE0	88
P7.1	89	P7.1/PM_CDOOUT/PM_TA0CLK	89
P7.2	90	P7.2/PM_C1OUT/PM_TA1CLK	90
P7.3	91	P7.3/PM_TA0.5	91
P7.4	26	P7.4/PM_TA1.4/CO.5	26
P7.5	27	P7.5/PM_TA1.3/CO.4	27
P7.6	28	P7.6/PM_TA1.2/CO.3	28
P7.7	29	P7.7/PM_TA1.1/CO.2	29

MSP1

P8.0/UCB3STE/TA1.0/CO.1	30	P8.0	30
P8.1/UCB3CLK/TA3.0/CO.0	31	P8.1	31
P8.2/TA3.2/A23	32	P8.2	32
P8.3/TA3CLK/A22	47	P8.3	47
P8.4/A21	48	P8.4	48
P8.5/A20	49	P8.5	49
P8.6/A19	50	P8.6	50
P8.7/A18	51	P8.7	51
P9.0/A17	52	P9.0	52
P9.1/A16	53	P9.1	53
P9.2	54	P9.2	54
P9.3/TA3.4	75	P9.3	75
P9.4/UCB3STE	96	P9.4	96
P9.5/UCB3CLK	97	P9.5	97
P9.6/UCB3RXD/UCB3SOMI	98	P9.6	98
P9.7/UCB3TXD/UCB3SIMO	99	P9.7	99
P10.0/UCB3STE	100	P10.0	100
P10.1/UCB3CLK	1	P10.1	1
P10.2/UCB3SIMQ/UCB3SDA	2	P10.2	2
P10.3/UCB3SOMI/UCB3SCL	3	P10.3	3
P10.4/TA3.0/CO.7	24	P10.4	24
P10.5/TA3.1/CO.6	25	P10.5	25
DCOR	44	DCOR	44
VSW	14	VSW	14
VCORE	12	VCORE	12
DVCC1	13	+3V3	13
DVCC2	73	+3V3	73
AVCC1	45	AVCC1	45
AVCC2	87	AVCC2	87
AVSS1	43	AVSS1	43
AVSS2	84	AVSS2	84
AVSS3	40	AVSS3	40
DVSS1	15	DVSS1	15
DVSS2	72	DVSS2	72
DVSS3	82	DVSS3	82
PJ.0/UFIXN	41	PJ.0/UFIXN	41
PJ.1/UFIXOUT	42	PJ.1/UFIXOUT	42
PJ.2/HFIXOUT	85	PJ.2/HFIXOUT	85
PJ.3/HFIXN	86	PJ.3/HFIXN	86
RSTN/NMI	83	RST	83
SWDIOTMS	94	TMS_SWDIO	94
SWCLKTCK	95	TCK_SWCLK	95
PJ.4/TDI/ADC14CLK	92	TDI	92
PJ.5/TDO/SWO	93	TDO_SWO	93

Target MCU:
MSP432P401R



COMPANY: Texas Instruments

TITLE: MSP-EXP432P401R_EagleCAD

DRAWN: Mike Stein

CHECKED:

APPROVED:

10/5/2016 5:02 PM

Rev: 2.1

Sheet: 1/6

MSP432

Sampling Voltage From PIN 5.5

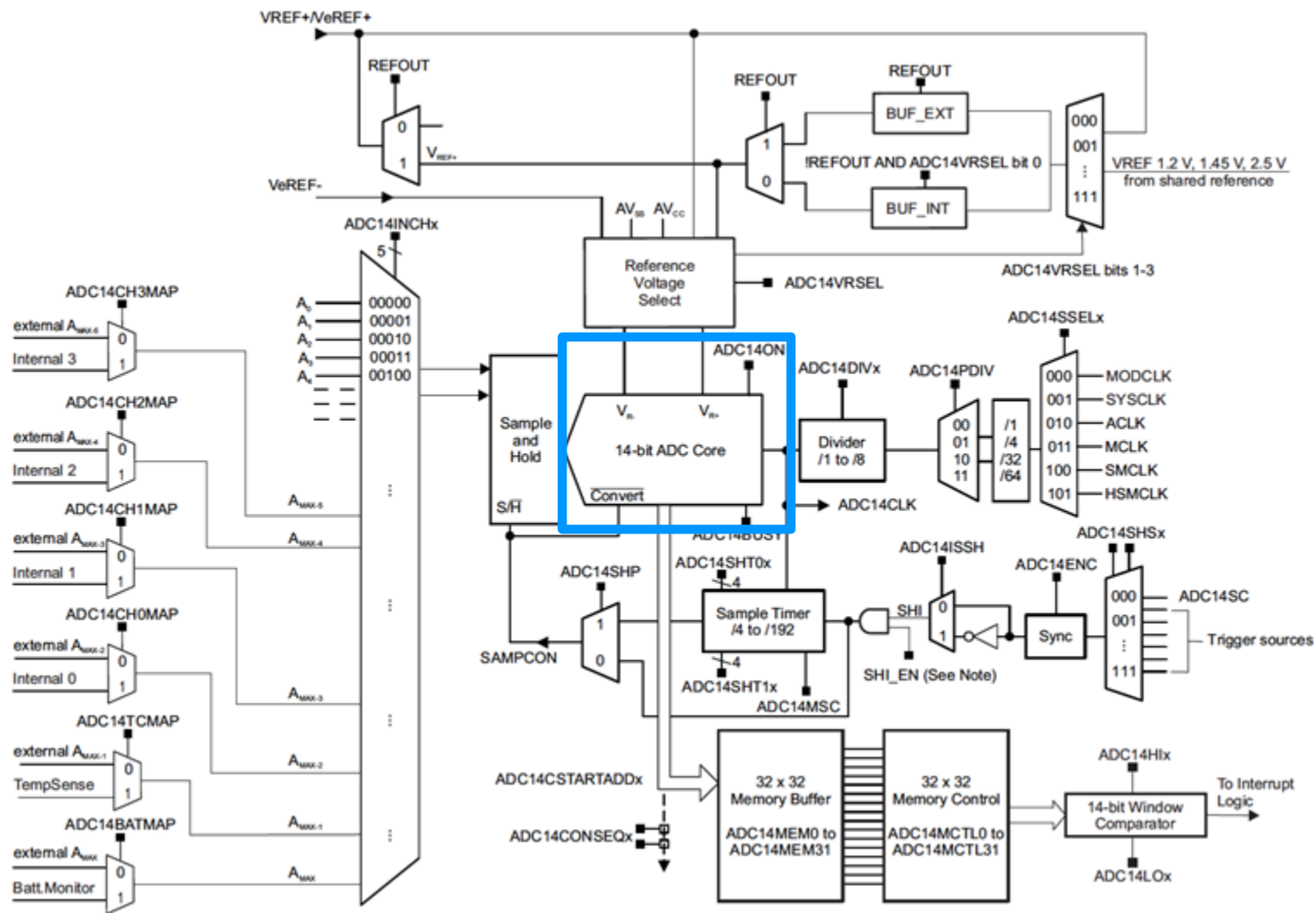
- Configure **Port 5.5** (A0) so that we use it as an input channel to ADC.

[illegible]

Sampling Voltage From PIN 5.5

- Enable ADC Hardware.

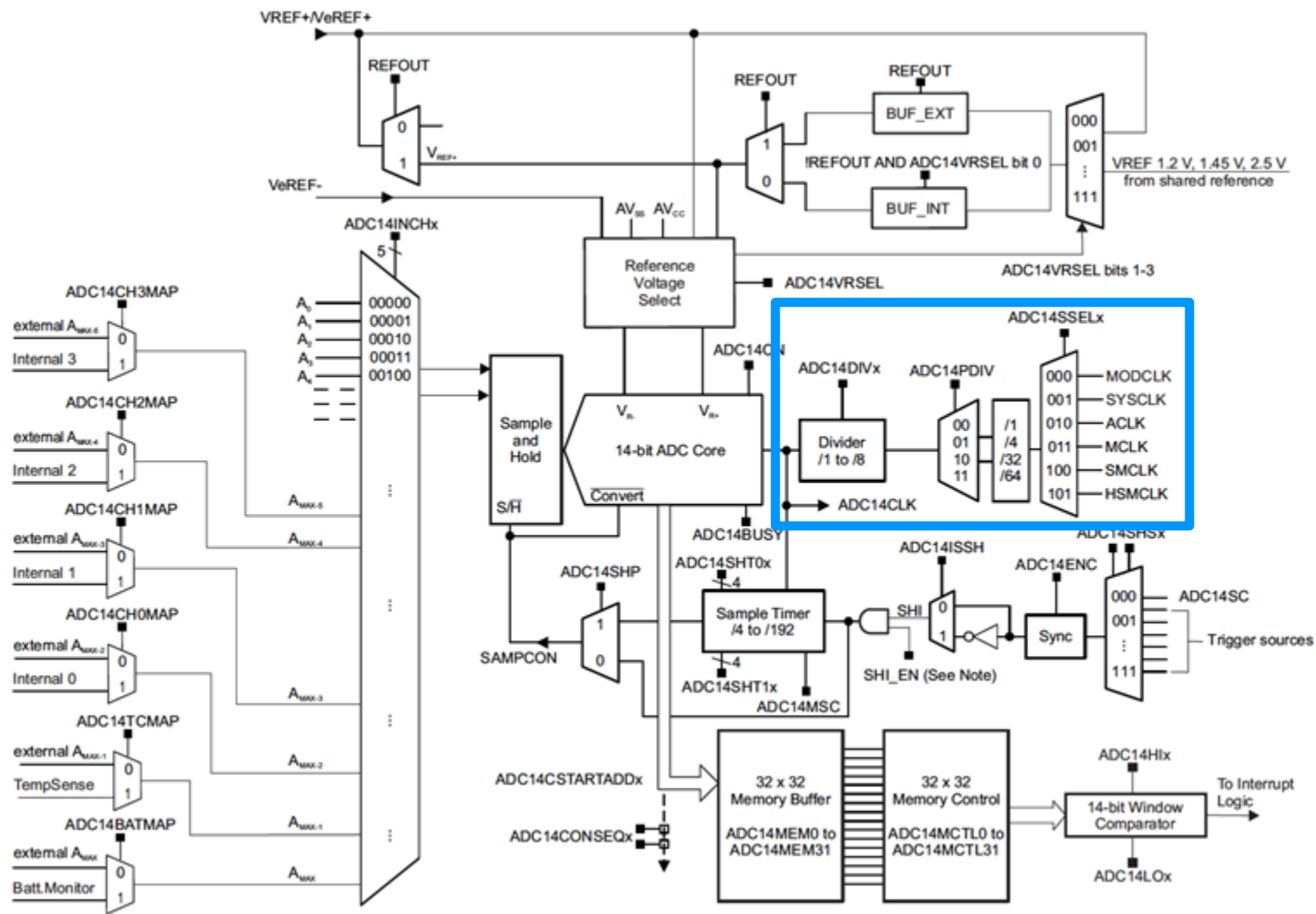
```
GPIO_setAsPeripheralModuleFunctionInputPin(GPIO_PORT_P5,GPIO_PIN5,  
                                             GPIO_TERTIARY_MODULE_FUNCTION);  
ADC14_enableModule();
```



Sampling Voltage From PIN 5.5

- Initializes the ADC module and sets up the clock system divider/pre-divider.
 - Set clock source as $MCLK/1/4 = 3MHz/4$

```
GPIO_setAsPeripheralModuleFunctionInputPin(GPIO_PORT_P5,GPIO_PIN5,  
GPIO_TERTIARY_MODULE_FUNCTION);  
ADC14_enableModule();  
ADC14_initModule(ADC_CLOCKSOURCE_MCLK, ADC_PREDIVIDER_1, ADC_DIVIDER_4,0);
```



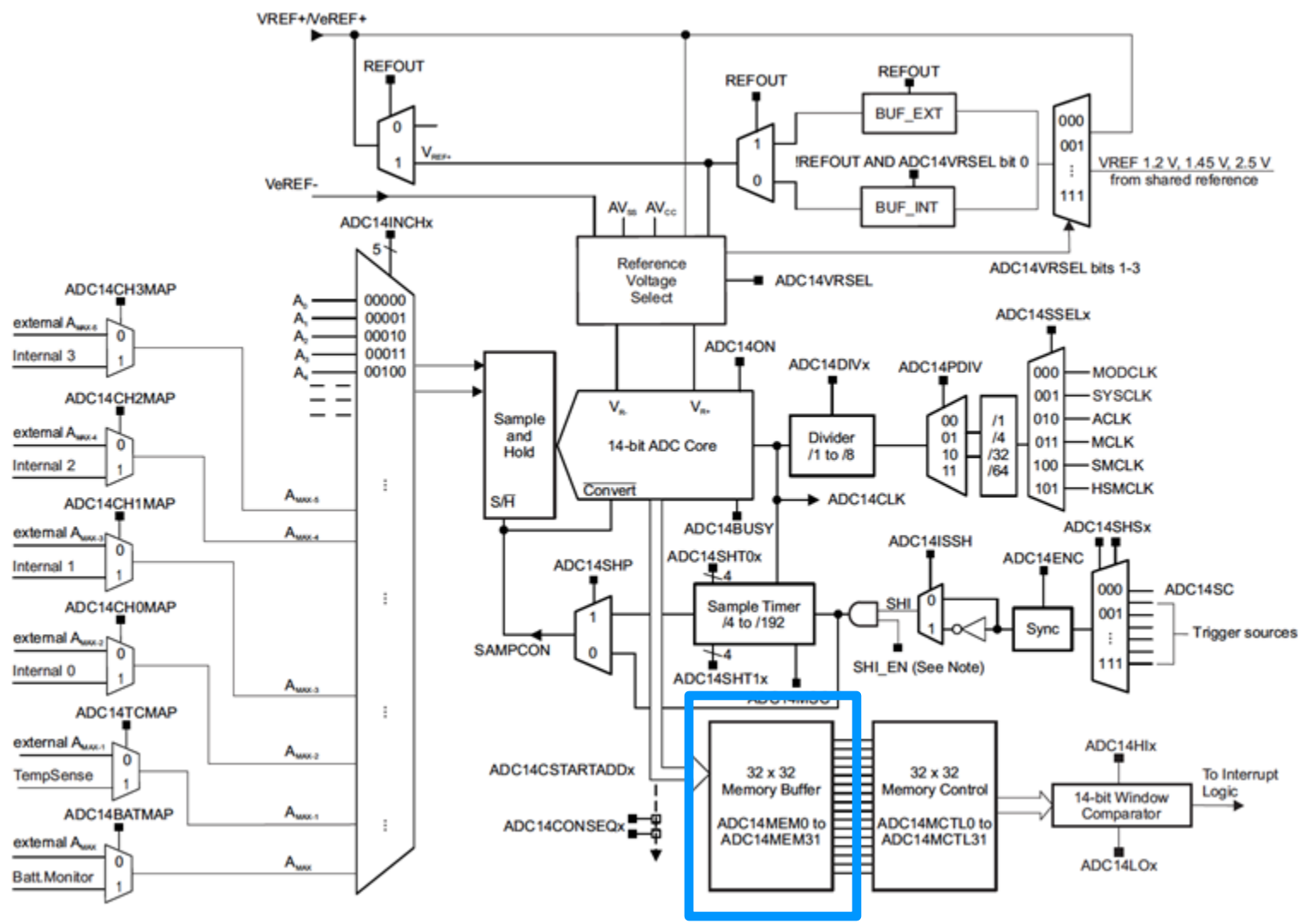
Sampling Voltage From PIN 5.5

- Use a single ADC memory location for sampling/conversion.

```
GPIO_setAsPeripheralModuleFunctionInputPin(GPIO_PORT_P5,GPIO_PIN5,  
GPIO_TERTIARY_MODULE_FUNCTION);  
ADC14_enableModule();  
ADC14_initModule(ADC_CLOCKSOURCE_MCLK, ADC_PREDIVIDER_1, ADC_DIVIDER_4,0);  
ADC14_configureSingleSampleMode(ADC_MEM0, true);
```



Specifies whether or not to **repeat** the conversion/sample cycle after the first round of sample/conversions



Sampling Voltage From PIN 5.5

- Configures an individual memory location for the ADC module.

```
GPIO_setAsPeripheralModuleFunctionInputPin(GPIO_PORT_P5,GPIO_PIN5,  
GPIO_TERTIARY_MODULE_FUNCTION);  
ADC14_enableModule();  
ADC14_initModule(ADC_CLOCKSOURCE_MCLK, ADC_PREDIVIDER_1, ADC_DIVIDER_4,0);  
ADC14_configureSingleSampleMode(ADC_MEM0, true);  
ADC14_configureConversionMemory(ADC_MEM0, ADC_VREFPOS_AVCC_VREFNEG_VSS,  
ADC_INPUT_A0, false);
```

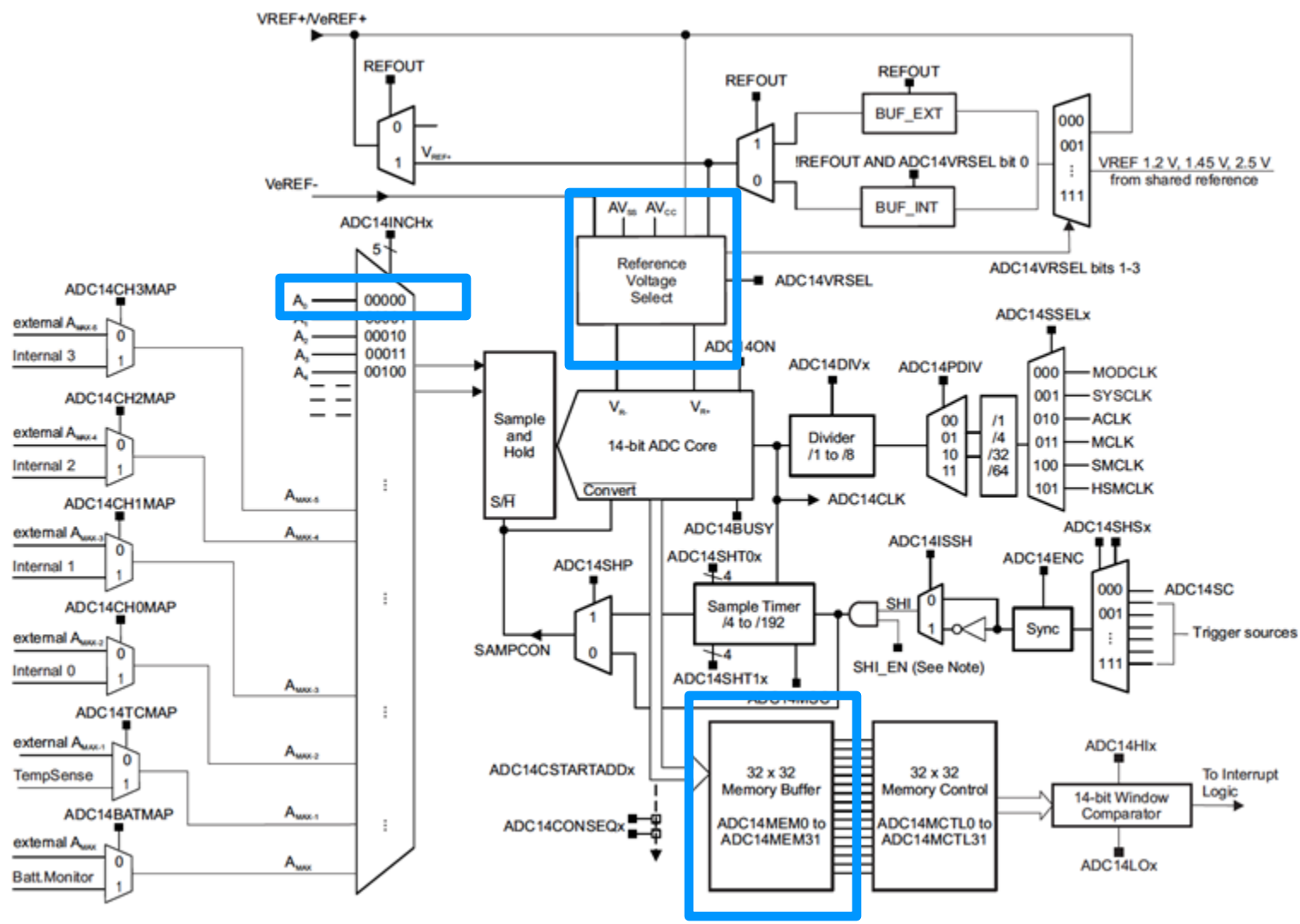
the channel to be
used for ADC
sampling

no
differential
mode.

memorySelect: is
the individual
ADC memory
location to
configure.

refSelect: is the
voltage reference
to use for the
selected memory
spot.

→ ADC_VREFPOS_AVCC_VREFNEG_VSS = 3.3V



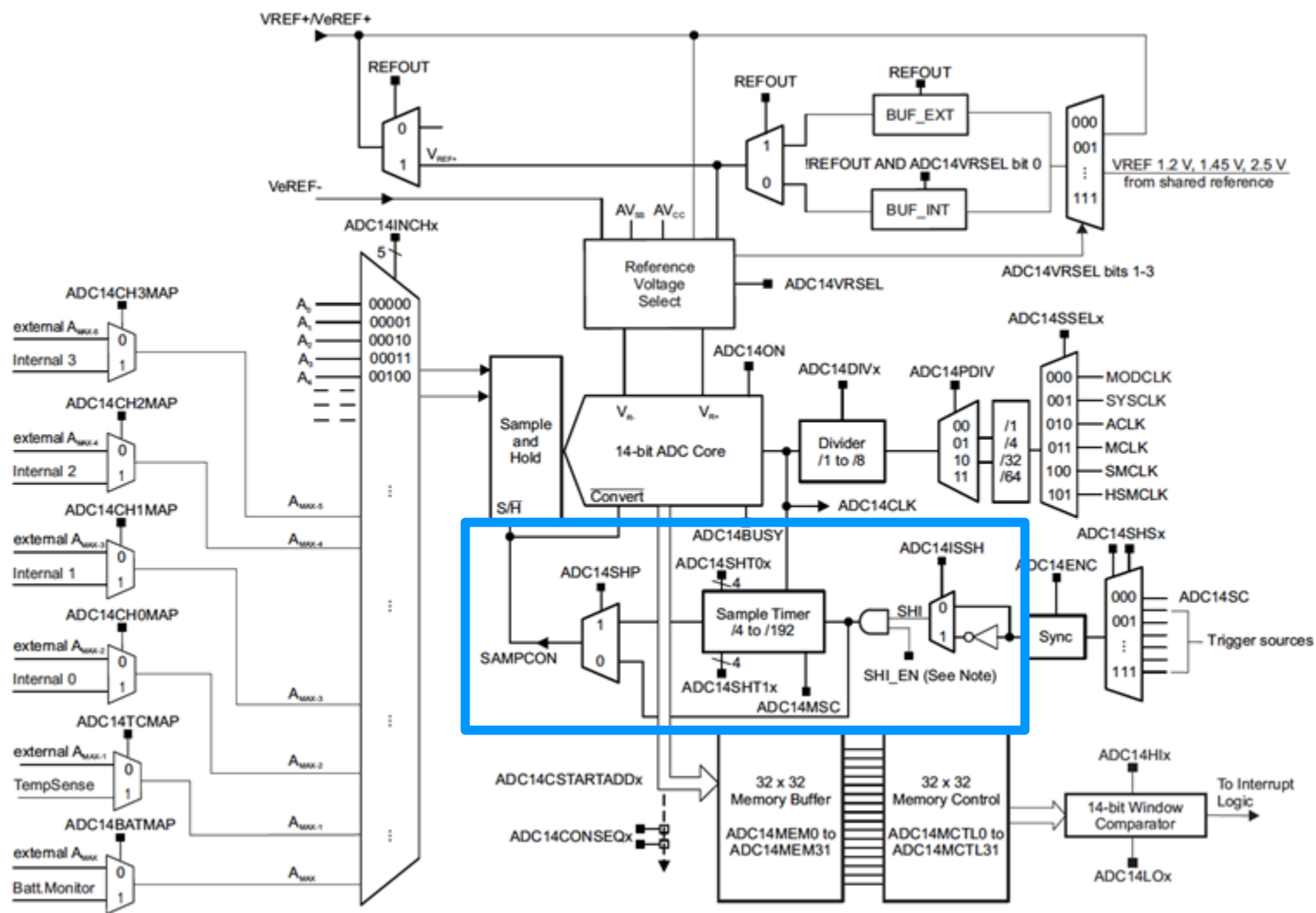
Sampling Voltage From PIN 5.5

- Enables SAMPCON to be sourced from the sampling timer

```
GPIO_setAsPeripheralModuleFunctionInputPin (GPIO_PORT_P5, GPIO_PIN5 ,  
GPIO_TERTIARY_MODULE_FUNCTION);  
ADC14_enableModule();  
ADC14_initModule (ADC_CLOCKSOURCE_MCLK, ADC_PREDIVIDER_1, ADC_DIVIDER_4, 0);  
ADC14_configureSingleSampleMode (ADC_MEM0, true);  
ADC14_configureConversionMemory (ADC_MEM0, ADC_VREFPOS_AVCC_VREFNEG_VSS,  
ADC_INPUT_A0, false);  
ADC14_enableSampleTimer (ADC_MANUAL_ITERATION);
```

The user will have to manually set the SHI signal (usually by *ADC14_toggleConversionTrigger*) at the end of each sample/conversion cycle.

SHI: Sample and Hold Indicator signal



Sampling Voltage From PIN 5.5

- Enable conversion of ADC data.
- Enables ADC to start conversion
- Enables ADC interrupts.

```
GPIO_setAsPeripheralModuleFunctionInputPin(GPIO_PORT_P5,GPIO_PIN5,  
GPIO_TERTIARY_MODULE_FUNCTION);  
ADC14_enableModule();  
ADC14_initModule(ADC_CLOCKSOURCE_MCLK, ADC_PREDIVIDER_1, ADC_DIVIDER_4,0);  
ADC14_configureSingleSampleMode(ADC_MEM0, true);  
ADC14_configureConversionMemory(ADC_MEM0, ADC_VREFPOS_AVCC_VREFNEG_VSS,  
ADC_INPUT_A0, false);  
ADC14_enableSampleTimer(ADC_MANUAL_ITERATION);  
ADC14_enableConversion();  
ADC14_toggleConversionTrigger();  
ADC14_enableInterrupt(ADC_INT0);  
Interrupt_enableInterrupt(INT_ADC14);  
Interrupt_enableMaster();
```

Sampling Voltage From PIN 5.5

- ADC Interrupt Service Routine

```
void ADC14_IRQHandler(void){
    uint64_t status = ADC14_getEnabledInterruptStatus();
    ADC14_clearInterruptFlag(status);

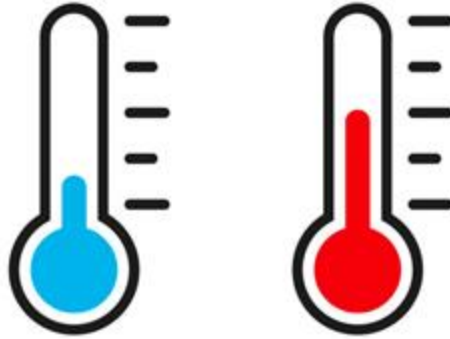
    if (ADC_INT0 & status){
        /* should be between 0 and 16384*/
        curADCResult = ADC14_getResult(ADC_MEM0);

        if(curADCResult > 16000)
            GPIO_setOutputHighOnPin(GPIO_PORT_P1,GPIO_PIN0);
        else
            GPIO_setOutputLowOnPin(GPIO_PORT_P1,GPIO_PIN0);

        /* next sample */
        ADC14_toggleConversionTrigger();
    }
}
```


Temperature Sensing

- Periodically sample values from the temperature sensor.
 - We will read voltage values using analog-to-digital converter.



Voltage Reference Module

- Generates reference voltages for use by various analog modules in a device.
 - 1.2 V, 1.45 V, and 2.5 V.
- Includes a **temperature sensor**
 - Can used by an ADC to measure a

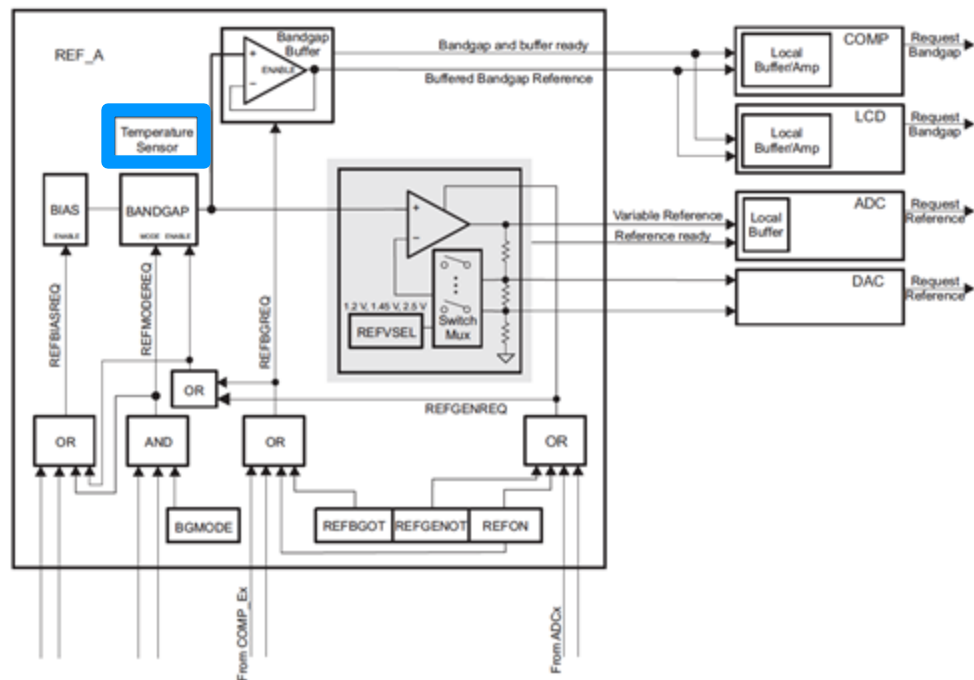


Figure 21-1. REF_A Block Diagram

On-chip Temperature Sensor

- Temperature sensor circuitry:
 - voltage proportional to temperature.

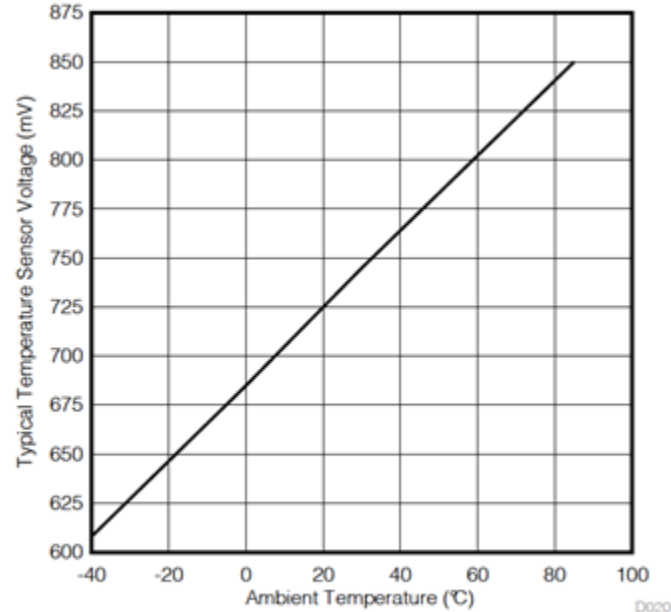


Figure 22-11. Typical Temperature Sensor Transfer Function

On-chip Temperature Sensor

- The temperature sensor is calibrated using the internal voltage references.
 - Each reference voltage (1.2V, 1.45V, or 2.5V) contains a measured value for two temperatures:
 - $30^{\circ}\text{C} \pm 3^{\circ}\text{C}$ and $85^{\circ}\text{C} \pm 3^{\circ}\text{C}$
- The temperature (Temp, $^{\circ}\text{C}$) can be computed as follows for each of the reference voltages used in the ADC measurement:

$$\text{Temp} = (\text{ADC}(\text{raw}) - \text{CAL_ADC_12T30}) \times \left(\frac{85 - 30}{\text{CAL_ADC_12T85} - \text{CAL_ADC_12T30}} \right) + 30$$

$$\text{Temp} = (\text{ADC}(\text{raw}) - \text{CAL_ADC_145T30}) \times \left(\frac{85 - 30}{\text{CAL_ADC_145T85} - \text{CAL_ADC_145T30}} \right) + 30$$

$$\text{Temp} = (\text{ADC}(\text{raw}) - \text{CAL_ADC_25T30}) \times \left(\frac{85 - 30}{\text{CAL_ADC_25T85} - \text{CAL_ADC_25T30}} \right) + 30$$

On-chip Temperature Sensor

- The temperature sensor is calibrated using the internal voltage references.
 - Each reference voltage (1.2V, 1.45V, or 2.5V) contains a measured value for two temperatures:
 - $30^{\circ}\text{C} \pm 3^{\circ}\text{C}$ and $85^{\circ}\text{C} \pm 3^{\circ}\text{C}$
- The temperature (Temp, $^{\circ}\text{C}$) can be computed as follows for each of the reference voltages used in the ADC measurement:

The diagram illustrates the formula for calculating temperature from ADC readings. It includes three versions of the formula, with callouts explaining specific values:

- Top formula:**
$$\text{Temp} = (\text{ADC}(\text{raw}) - \text{CAL_ADC_25T30}) \times \left(\frac{85 - 30}{\text{CAL_ADC_145T85} - \text{CAL_ADC_145T30}} \right) + 30$$
 - Callout 1 (left): "The ADC reading(with 2.5V Reference) that corresponds to the temperature value 30" points to CAL_ADC_25T30 .
 - Callout 2 (right): "The ADC reading(with 2.5V Reference) that corresponds to the temperature value 80" points to CAL_ADC_145T85 .
- Middle formula:**
$$\text{Temp} = (\text{ADC}(\text{raw}) - \text{CAL_ADC_25T30}) \times \left(\frac{85 - 30}{\text{CAL_ADC_145T85} - \text{CAL_ADC_145T30}} \right) + 30$$
- Bottom formula:**
$$\text{Temp} = (\text{ADC}(\text{raw}) - \text{CAL_ADC_25T30}) \times \left(\frac{85 - 30}{\text{CAL_ADC_25T85} - \text{CAL_ADC_25T30}} \right) + 30$$
 - Callout 3 (bottom): "The ADC reading(with 2.5V Reference) that corresponds to the temperature value 80" points to CAL_ADC_25T85 .

On-chip Temperature Sensor

```
/* Setting reference voltage to 2.5 and  
enabling temperature sensor */  
REF_A_enableTempSensor();  
REF_A_setReferenceVoltage(REF_A_VREF2_5V);  
REF_A_enableReferenceVoltage();
```

On-chip Temperature Sensor

```
/* Setting reference voltage to 2.5 and  
enabling temperature sensor */  
REF_A_enableTempSensor();  
REF_A_setReferenceVoltage(REF_A_VREF2_5V);  
REF_A_enableReferenceVoltage();
```

```
void REF_A_enableTempSensor ( void )
```

Enables the internal temperature sensor.

This function is used to turn on the internal temperature sensor to use by other peripherals. The temperature sensor is enabled by default. Please note, if the [REF_A_isRefGenBusy\(\)](#) returns **REF_A_BUSY**, this function will have no effect.

Modified bits are **REFTCOFF** of **REFCTL0** register.

Returns

none

On-chip Temperature Sensor

```
/* Setting reference voltage to 2.5 and  
enabling temperature sensor */  
REF_A_enableTempSensor();  
REF_A_setReferenceVoltage(REF_A_VREF2_5V);  
REF_A_enableReferenceVoltage();
```

```
void REF_A_setReferenceVoltage ( uint_fast8_t referenceVoltageSelect )
```

Sets the reference voltage for the voltage generator.

Parameters

<i>referenceVoltageSelect</i>	is the desired voltage to generate for a reference voltage. Valid values are: ■ REF_A_VREF1_2V [Default] ■ REF_A_VREF1_45V ■ REF_A_VREF2_5V Modified bits are REFVSEL of REFCTL0 register.
-------------------------------	--

This function sets the reference voltage generated by the voltage generator to be used by other peripherals. This reference voltage will only be valid while the REF module is in control. Please note, if the [REF_A_isRefGenBusy\(\)](#) returns **REF_BUSY**, this function will have no effect.

Returns

none

On-chip Temperature Sensor

```
/* Setting reference voltage to 2.5 and  
enabling temperature sensor */  
REF_A_enableTempSensor();  
REF_A_setReferenceVoltage(REF_A_VREF2_5V);  
REF_A_enableReferenceVoltage();
```

```
void REF_A_enableReferenceVoltage ( void )
```

Enables the reference voltage to be used by peripherals.

This function is used to enable the generated reference voltage to be used other peripherals or by an output pin, if enabled. Please note, that giving ADC12 module control over the REF module, the state of the reference voltage is dependent on the controls of the ADC12 module. Please note, if the [REF_A_isRefGenBusy\(\)](#) returns REF_A_BUSY, this function will have no effect.

Modified bits are **REFON** of **REFCTL0** register.

On-chip Temperature Sensor

- We need to know which voltage values are used for 30 and 85 degrees

```
uint32_t cal30;  
uint32_t cal85;  
float calDifference;  
...  
  
cal30 = SysCtl_getTempCalibrationConstant(SYSCTL_2_5V_REF, SYSCTL_30_DEGREES_C);  
cal85 = SysCtl_getTempCalibrationConstant(SYSCTL_2_5V_REF, SYSCTL_85_DEGREES_C);  
calDifference = cal85 - cal30;
```

On-chip Temperature Sensor

- We need to know which voltage value

```
uint32_t cal30;  
uint32_t cal85;  
float calDifference;  
...
```

```
cal30 = SysCtl_getTempCalibrationConstant(SYSCTL_2_5V_REF, SYSCTL_30_DEGREES_C);  
cal85 = SysCtl_getTempCalibrationConstant(SYSCTL_2_5V_REF, SYSCTL_85_DEGREES_C);  
calDifference = cal85 - cal30;
```

uint_fast16_t SysCtl_getTempCalibrationConstant (uint32_t refVoltage, uint32_t temperature)

Retrieves the calibration constant of the temperature sensor to be used in temperature calculation.

Parameters

refVoltage	Reference voltage being used.
------------	-------------------------------

The *refVoltage* parameter must be only one of the following values:

- SYSCTL_1_2V_REF
- SYSCTL_1_45V_REF
- SYSCTL_2_5V_REF

Parameters

temperature	is the calibration temperature that the user wants to be returned.
-------------	--

The *temperature* parameter must be only one of the following values:

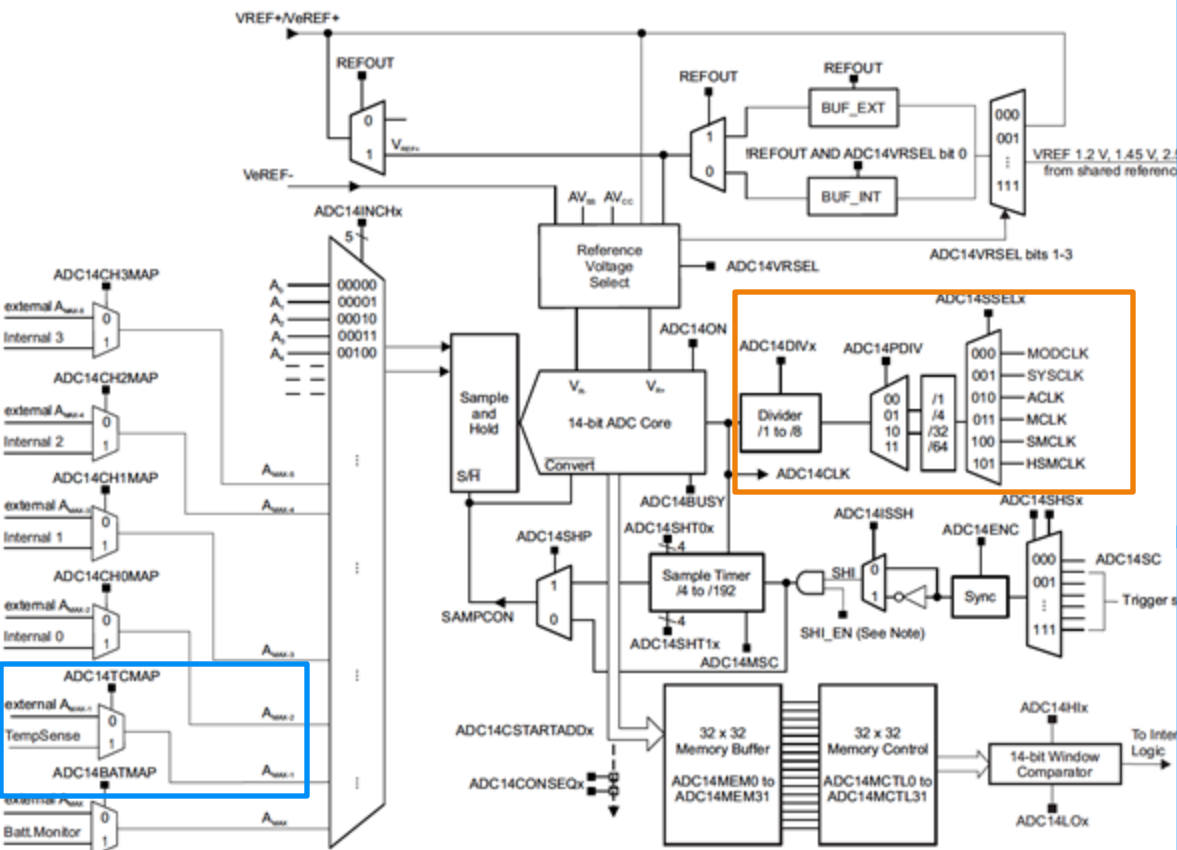
- SYSCTL_30_DEGREES_C
- SYSCTL_85_DEGREES_C

Returns

None.

On-chip Temperature Sensor

```
/* Initializing ADC (MCLK/1/1) with temperature sensor routed */  
ADC14_enableModule();  
ADC14_initModule(ADC_CLOCKSOURCE_MCLK, ADC_PREDIVIDER_1,  
ADC_DIVIDER_1, ADC_TEMPSENSEMAP);
```



bool ADC14_initModule(uint32_t clockSource, uint32_t clockPredivider, uint32_t clockDivider, uint32_t internalChannelMask)

Initializes the ADC module and sets up the clock system divider/pre-divider. This initialization function will also configure the internal/external signal mapping.

Note

A call to this function while active ADC conversion is happening is an invalid case and will result in a false value being returned.

Parameters

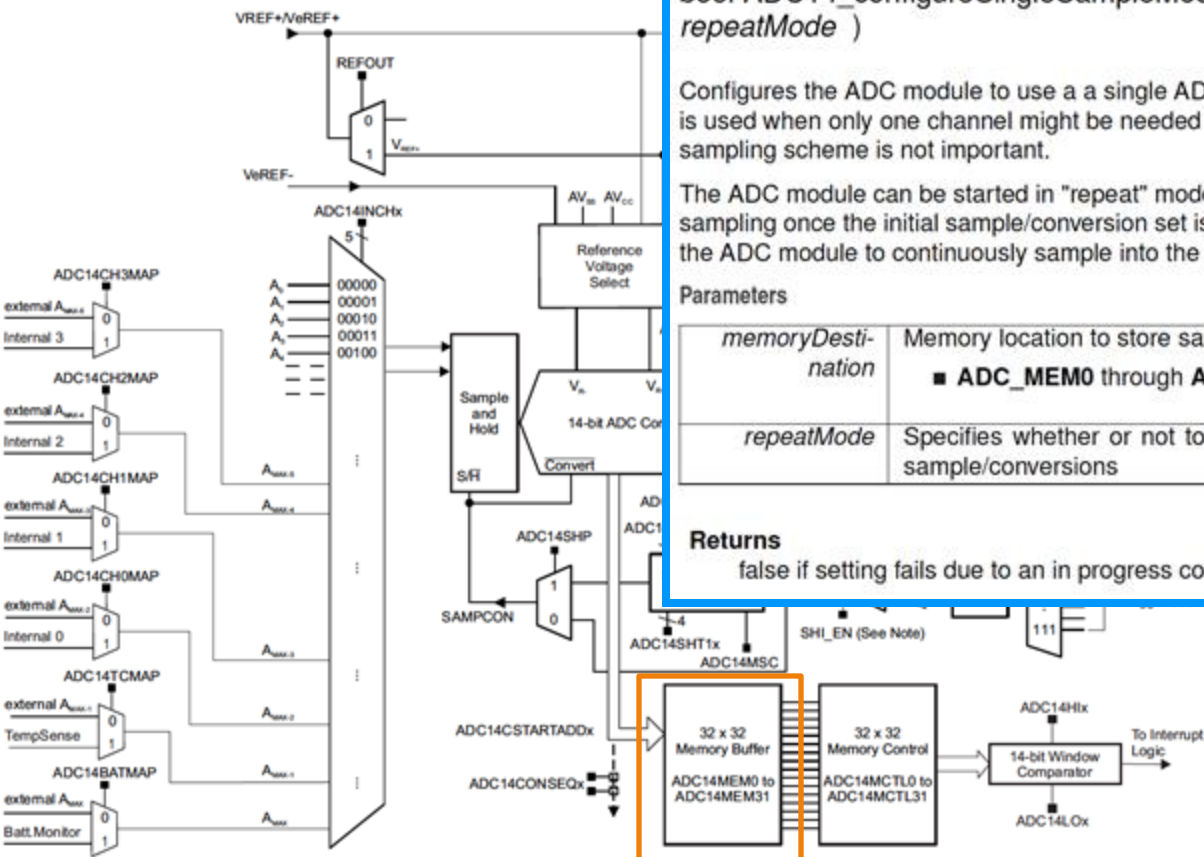
clockSource	The clock source to use for the ADC module. ■ ADC_CLOCKSOURCE_ADCOSC [DEFAULT] ■ ADC_CLOCKSOURCE_SYSOSC ■ ADC_CLOCKSOURCE_ACLK ■ ADC_CLOCKSOURCE_MCLK ■ ADC_CLOCKSOURCE_SMCLK ■ ADC_CLOCKSOURCE_HSMCLK
clockPredivider	Divides the given clock source before feeding it into the main clock divider. Valid values are: ■ ADC_PREDIVIDER_1 [DEFAULT] ■ ADC_PREDIVIDER_4 ■ ADC_PREDIVIDER_32 ■ ADC_PREDIVIDER_64
clockDivider	Divides the pre-divided clock source. Valid values are: ■ ADC_DIVIDER_1 [Default value] ■ ADC_DIVIDER_2 ■ ADC_DIVIDER_3 ■ ADC_DIVIDER_4 ■ ADC_DIVIDER_5 ■ ADC_DIVIDER_6 ■ ADC_DIVIDER_7 ■ ADC_DIVIDER_8
internalChannelMask	Configures the internal/external pin mappings for the ADC modules. This setting determines if the given ADC channel or component is mapped to an external pin (default), or routed to an internal component. This parameter is a bit mask where a logical high value will switch the component to the internal routing. For a list of internal routings, please refer to the device specific data sheet. Valid values are a logical OR of the following values: ■ ADC_MAPINTCH3 ■ ADC_MAPINTCH2 ■ ADC_MAPINTCH1 ■ ADC_MAPINTCH0 ■ ADC_TEMPSENSEMAP ■ ADC_BATTMAP ■ ADC_NOROUTE If internalChannelMask is not desired, pass ADC_NOROUTE in lieu of this parameter.

Returns

false if the initialization fails due to an in progress conversion

On-chip Temperature Sensor

```
/* Initializing ADC (MCLK/1/1) with temperature sensor routed */
ADC14_enableModule();
ADC14_initModule(ADC_CLOCKSOURCE_MCLK, ADC_PREDIVIDER_1,
ADC_DIVIDER_1, ADC_TEMPSENSEMAP);
/* Configuring ADC Memory (ADC_MEM0 A22 (Temperature Sensor) in repeat mode */
ADC14_configureSingleSampleMode(ADC_MEM0, true);
ADC14_configureConversionMemory(ADC_MEM0,
ADC_VREFPOS_INTBUF_VREFNEG_VSS, ADC_INPUT_A22, false);
```



```
bool ADC14_configureSingleSampleMode ( uint32_t memoryDestination, bool repeatMode )
```

Configures the ADC module to use a single ADC memory location for sampling/conversion. This is used when only one channel might be needed for conversion, or where using a multiple sampling scheme is not important.

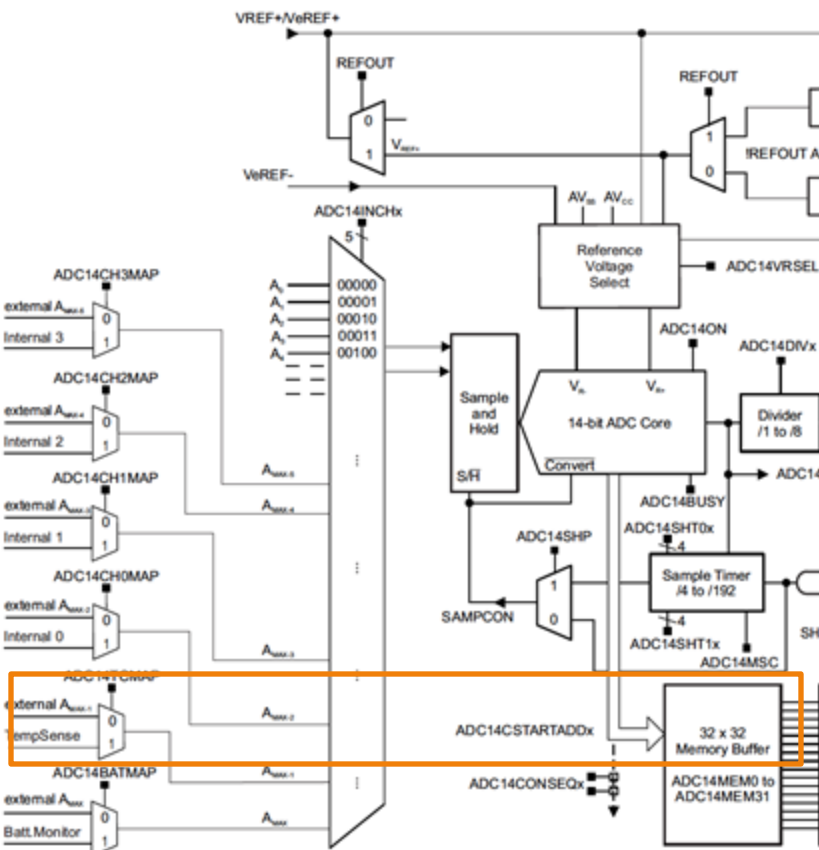
The ADC module can be started in "repeat" mode which will cause the ADC module to resume sampling once the initial sample/conversion set is executed. In single sample mode, this will cause the ADC module to continuously sample into the memory destination provided.

Parameters

<i>memoryDestination</i>	Memory location to store sample/conversion value. Possible values include: ■ ADC_MEM0 through ADC_MEM31
<i>repeatMode</i>	Specifies whether or not to repeat the conversion/sample cycle after the first round of sample/conversions

Returns

false if setting fails due to an in progress conversion



```
bool ADC14_configureConversionMemory ( uint32_t memorySelect, uint32_t refSelect, uint32_t channelSelect, bool differentialMode )
```

Configures an individual memory location for the ADC module.

Parameters

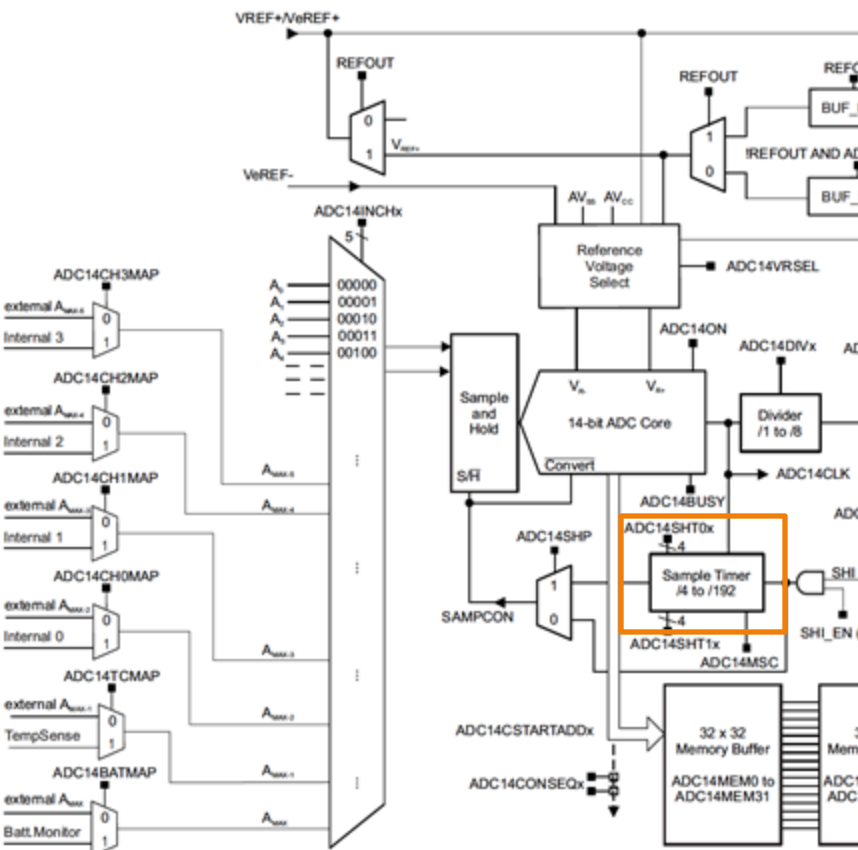
<i>memorySelect</i>	is the individual ADC memory location to configure. If multiple memory locations want to be configured with the same configuration, this value can be logically ORed together with other values. ■ ADC_MEM0 through ADC_MEM31
<i>refSelect</i>	is the voltage reference to use for the selected memory spot. Possible values include: ■ ADC_VREFPOS_AVCC_VREFNEG_VSS [DEFAULT] ■ ADC_VREFPOS_INTBUF_VREFNEG_VSS ■ ADC_VREFPOS_EXTPOS_VREFNEG_EXTNEG ■ ADC_VREFPOS_EXTBUF_VREFNEG_EXTNEG
<i>channelSelect</i>	selects the channel to be used for ADC sampling. Note if differential mode is enabled, the value sampled will be equal to the difference between the corresponding even/odd memory locations. Possible values are: ■ ADC_INPUT_A0 through ADC_INPUT_A31
<i>differentialMode</i>	selects if the channel selected by the channelSelect will be configured in differential mode. If this parameter is given as true, the configured channel will be paired with its neighbor in differential mode. for example, if channel A0 or A1 is selected, the channel configured will be the difference between A0 and A1. If A2 or A3 are selected, the channel configured will be the difference between A2 and A3 (and so on). Users can enter true or false, or one of the following values: ■ ADC_NONDIFFERENTIAL_INPUTS ■ ADC_DIFFERENTIAL_INPUTS

Returns

false if setting fails due to an in progress conversion

On-chip Temperature Sensor

```
/* Initializing ADC (MCLK/1/1) with temperature sensor routed */
ADC14_enableModule();
ADC14_initModule(ADC_CLOCKSOURCE_MCLK, ADC_PREDIVIDER_1,
ADC_DIVIDER_1, ADC_TEMPSENSEMAP);
/* Configuring ADC Memory (ADC_MEM0 A22 (Temperature Sensor) in repeat mode */
ADC14_configureSingleSampleMode(ADC_MEM0, true);
ADC14_configureConversionMemory(ADC_MEM0,
ADC_VREFPOS_INTBUF_VREFNEG_VSS, ADC_INPUT_A22, false);
/* Configuring the sample/hold time for 192 */
ADC14_setSampleHoldTime(ADC_PULSE_WIDTH_192, ADC_PULSE_WIDTH_192);
```



```
bool ADC14_setSampleHoldTime ( uint32_t firstPulseWidth, uint32_t secondPulseWidth )
```

Sets the sample/hold time for the specified memory register range. The duration of time required for a sample differs depending on the user's hardware configuration.

There are two values in the ADCC module. The first value controls ADC memory locations ADC_MEMORY_0 through ADC_MEMORY_7 and ADC_MEMORY_24 through ADC_MEMORY_31, while the second value controls memory locations ADC_MEMORY_8 through ADC_MEMORY_23.

Parameters

<i>firstPulseWidth</i>	Pulse width of the first pulse in ADCCLK cycles. Possible values must be one of the following: ■ ADC_PULSE_WIDTH_4 [DEFAULT] ■ ADC_PULSE_WIDTH_8 ■ ADC_PULSE_WIDTH_16 ■ ADC_PULSE_WIDTH_32 ■ ADC_PULSE_WIDTH_64 ■ ADC_PULSE_WIDTH_96 ■ ADC_PULSE_WIDTH_128 ■ ADC_PULSE_WIDTH_192
<i>secondPulseWidth</i>	Pulse width of the second pulse in ADCCLK cycles. Possible values must be one of the following: ■ ADC_PULSE_WIDTH_4 [DEFAULT] ■ ADC_PULSE_WIDTH_8 ■ ADC_PULSE_WIDTH_16 ■ ADC_PULSE_WIDTH_32 ■ ADC_PULSE_WIDTH_64 ■ ADC_PULSE_WIDTH_96 ■ ADC_PULSE_WIDTH_128 ■ ADC_PULSE_WIDTH_192

Returns

false if setting fails due to an in progress conversion

On-chip Temperature Sensor

```
/* Initializing ADC (MCLK/1/1) with temperature sensor routed */
ADC14_enableModule();
ADC14_initModule(ADC_CLOCKSOURCE_MCLK, ADC_PREDIVIDER_1,
ADC_DIVIDER_1, ADC_TEMPSENSEMAP);
/* Configuring ADC Memory (ADC_MEM0 A22 (Temperature Sensor) in repeat mode */
ADC14_configureSingleSampleMode(ADC_MEM0, true);
ADC14_configureConversionMemory(ADC_MEM0,
ADC_VREFPOS_INTBUF_VREFNEG_VSS, ADC_INPUT_A22, false);
/* Configuring the sample/hold time for 192 */
ADC14_setSampleHoldTime(ADC_PULSE_WIDTH_192, ADC_PULSE_WIDTH_192);
/* Enabling sample timer in auto iteration mode and interrupts*/
ADC14_enableSampleTimer(ADC_AUTOMATIC_ITERATION);
```


On-chip Temperature Sensor

```
void ADC14_IRQHandler(void) {
    uint64_t status;
    int16_t conRes;

    status = ADC14_getEnabledInterruptStatus();
    ADC14_clearInterruptFlag(status);

    if(status & ADC_INT0){
        conRes = ((ADC14_getResult(ADC_MEM0) - cal30) * 55);
        tempC = (conRes / calDifference) + 30.0f;
        tempF = tempC * 9.0f / 5.0f + 32.0f;
    }
}
```

Lab #1: Periodic Temperature Sensing

- Implement the following FSM **using DriverLib**.
 - Sample temperature **every 1 second** at transition to the corresponding state.

